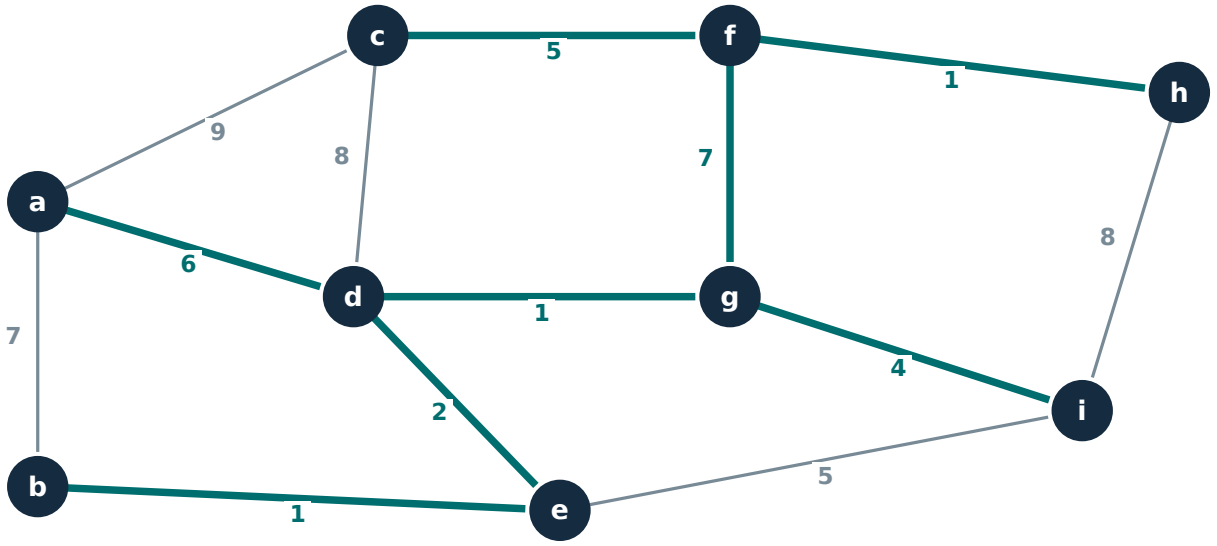
Algoritmos de grafos

GUIA VISUAL • CONCEITO → REGRA → EXECUÇÃO

Dez algoritmos explicados com desenhos, tabelas, pseudocódigo e decisões justificadas.



As arestas verdes formam uma árvore geradora mínima de custo 27. O guia mostra como chegar a essa escolha.

> **Feito para acompanhar seu material**
>
> A sequência e os conceitos seguem os slides de Zenilton, com apoio dos slides de Silvio e do livro de Douglas B. West disponível na pasta. Os desenhos foram refeitos e os exemplos executados para este guia.

Prim • Kruskal • Dijkstra • Bellman-Ford • Floyd-Warshall • Ford-Fulkerson • Edmonds-Karp • Dinic • Fleury • Kosaraju

Leia o conceito, cubra a próxima etapa e tente decidir sozinho. Depois confira a aresta escolhida, a conta e o estado atualizado.

Roteiro de leitura

Os números abaixo são as páginas deste guia. Clique em uma linha ou use os marcadores do leitor de PDF para saltar de seção.

Qual problema você quer resolver? **3**

Prim: crescer a partir de uma raiz **6**

Kruskal: unir componentes sem ciclos **13**

Dijkstra: fechar a menor distância provisória **18**

Bellman-Ford: repetir as relaxações **24**

Floyd-Warshall: permitir intermediários **31**

Capacidade, conservação e valor do fluxo **36**

Ford-Fulkerson: aumentar enquanto for possível **38**

Edmonds-Karp: o caminho com menos arestas **42**

Dinic: um fluxo de bloqueio por fase **45**

Fleury: passar sem se prender **49**

Kosaraju: duas DFS separam os grupos **54**

Uma frase para lembrar de cada algoritmo **60**

Onde este guia se apoia **62**

Qual problema você quer resolver?

Pergunta	Algoritmo	O que a resposta representa
Conectar todos com menor custo total?	Prim / Kruskal	Uma árvore geradora mínima (AGM).
Distância de uma origem aos outros?	Dijkstra	Caminhos mínimos; pesos não negativos.
Há pesos negativos?	Bellman-Ford	Caminhos de uma origem; detecta ciclo negativo alcançável.
Distâncias entre todos os pares?	Floyd-Warshall	Uma matriz de distâncias.
Quanto pode atravessar uma rede?	Ford-Fulkerson / Edmonds-Karp / Dinic	Um fluxo máximo da fonte ao sumidouro.
Usar cada aresta exatamente uma vez?	Fleury	Trilha ou circuito euleriano.
Quem se alcança nos dois sentidos?	Kosaraju	Componentes fortemente conexos.

A pergunta vem antes do algoritmo

Uma AGM minimiza a soma das arestas da árvore. Um caminho mínimo minimiza a soma ao longo de um percurso. Um fluxo máximo combina percursos e respeita capacidades.

Como ler os desenhos e as tabelas

• G = (V,E): V é o conjunto de vértices e E é o de arestas. n = |V| e m = |E|.

• Linha sem seta: aresta não direcionada. Uma seta u→v permite ir de u para v; o sentido contrário exige outra aresta.

• w(u,v) é o peso/custo. c(u,v), também chamado u(e) nos slides de fluxo, é a capacidade. Custo e capacidade têm papéis diferentes.

• d[v] ou dist[v] é a melhor distância conhecida. ∞ significa que ainda não encontramos um caminho.

• pred[v] ou π[v] guarda o vértice anterior no caminho. S costuma guardar os vértices já selecionados.

• Verde: arestas escolhidas. Laranja: decisão atual. Tracejadas claras: arestas já retiradas. O texto também identifica cada decisão.

Para reproduzir as contas

Os grafos principais são os dos slides indicados. Empates usam ordem alfabética ou numérica, salvo indicação. A ordem pode diferir das animações, mas a regra do algoritmo e a validade do resultado são mantidas.

> **BFS e DFS**
>
> BFS usa fila: visita primeiro os vértices com menos arestas até a origem. DFS usa recursão ou pilha: aprofunda um ramo antes de voltar. Kosaraju usa DFS; Edmonds-Karp e os níveis de Dinic usam BFS.

Relaxar: a conta que melhora um caminho

Já conhecemos uma rota até u de custo 5. A aresta u→v custa 2. Então existe uma rota até v de custo 5 + 2 = 7. Se a estimativa antiga era 10, encontramos uma melhora.



Regra de relaxação

Se d[u] for finita e d[v] > d[u] + w(u,v), faça d[v] = d[u] + w(u,v) e pred[v] = u. Se o custo empatar, pode manter o predecessor anterior.

Algoritmo	Quem controla a comparação?
Dijkstra	O vértice não finalizado com menor d[v].
Bellman-Ford	Varreduras repetidas por todas as arestas.
Floyd-Warshall	Um intermediário k: compare i→j com i→k→j.

Relaxar não é escolher apenas a aresta mais barata. O custo acumulado até a origem também entra na conta. Essa diferença explica por que Dijkstra e Prim não são intercambiáveis.

Prim: crescer a partir de uma raiz

Objetivo: conectar todos os vértices usando n−1 arestas, sem ciclos, com a menor soma de pesos. A entrada é um grafo não direcionado, ponderado e conexo. Pesos negativos são permitidos.

> **A regra dos slides**
>
> Mantenha os vértices incluídos em V(T). Escolha a aresta de menor peso com uma ponta dentro de V(T) e outra fora. Inclua a aresta e o novo vértice.

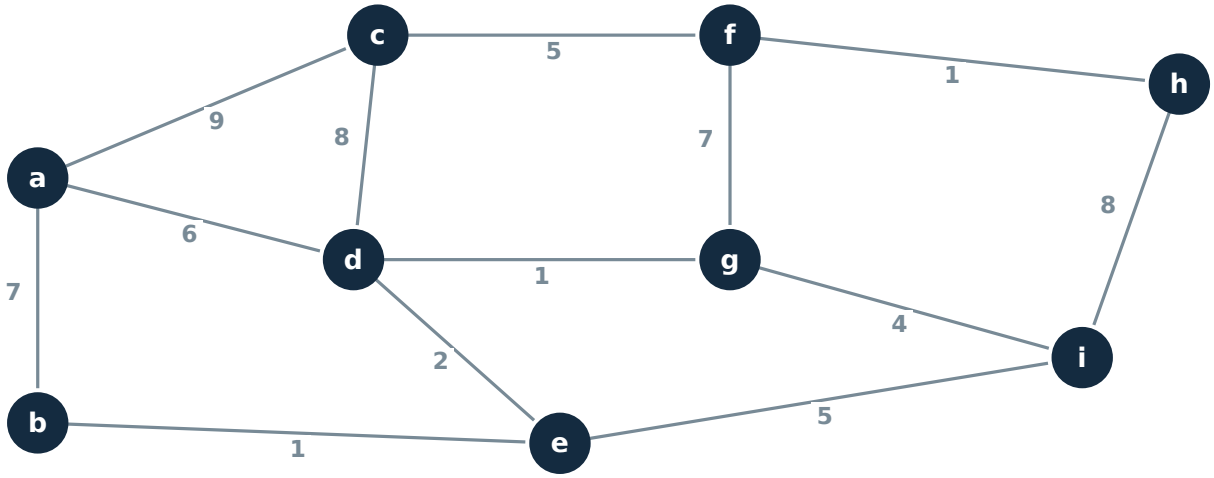
Pense em expandir uma rede já construída. Só interessam ligações da parte pronta até um vértice novo. Uma aresta barata com ambas as pontas fora ainda não é candidata.

```
escolha uma raiz r
S = {r}; T = conjunto vazio
enquanto S não contiver todos os vértices:
    escolha (u,v) de menor peso cruzando S e V−S
    acrescente (u,v) a T
    acrescente o vértice de fora a S
retorne T
```

Se faltarem vértices e não existir aresta cruzando o corte, o grafo é desconexo. Reinicie em outra componente para obter uma floresta geradora mínima. Não existe árvore geradora cobrindo um grafo desconexo.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

O grafo da aula e as oito escolhas



Grafo do slide 11. Começamos em a. Há 9 vértices, portanto a árvore terá 8 arestas.

Passo	Aresta / peso	Novo vértice	Total
1	a—d / 6	d	6
2	d—g / 1	g	7
3	d—e / 2	e	9
4	b—e / 1	b	10
5	g—i / 4	i	14
6	f—g / 7	f	21
7	f—h / 1	h	22
8	c—f / 5	c	27

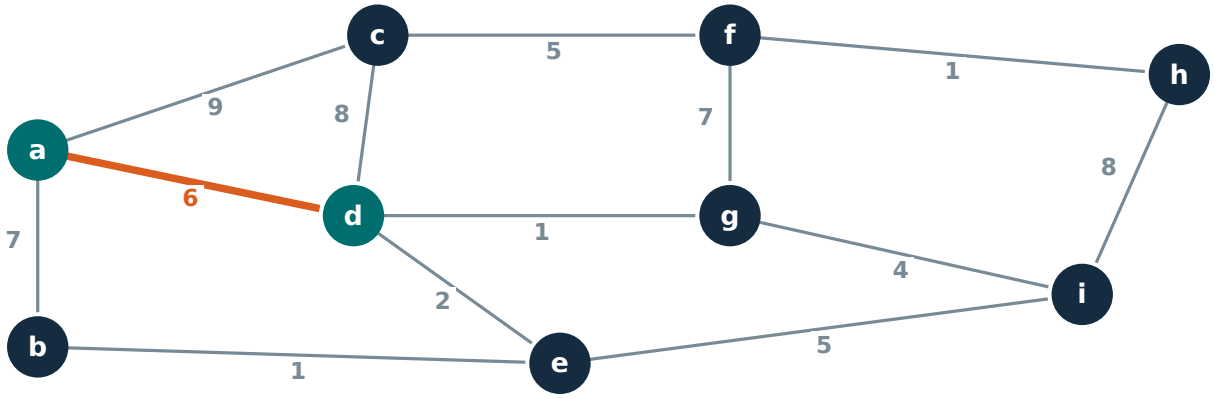
Resultado

T = {a—d, d—g, d—e, b—e, g—i, f—g, f—h, c—f}. Soma = 6 + 1 + 2 + 1 + 4 + 7 + 1 + 5 = 27.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

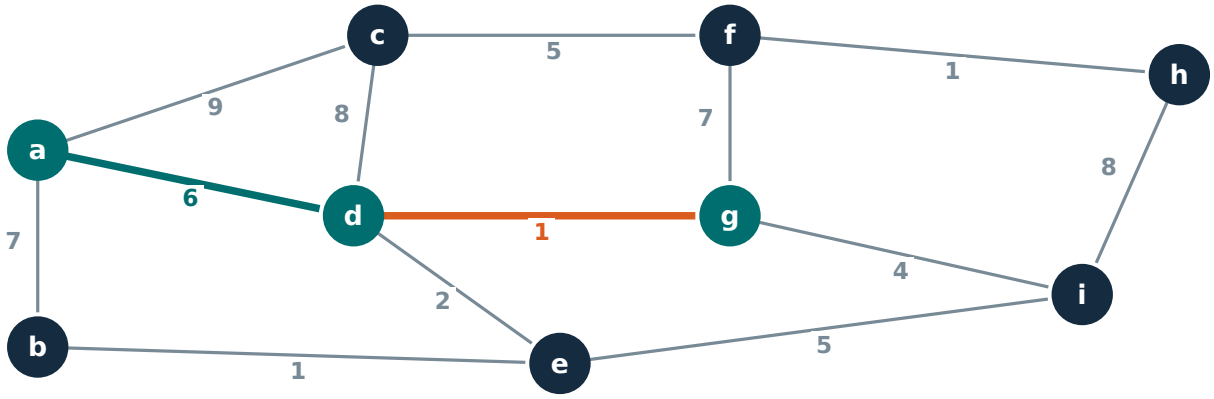
Execução visual • passos 1 e 2

1. Escolha a—d, peso 6 | total 6



S={a}. Compare a—d=6, a—b=7 e a—c=9. A menor saída é a—d; inclua d.

2. Escolha d—g, peso 1 | total 7

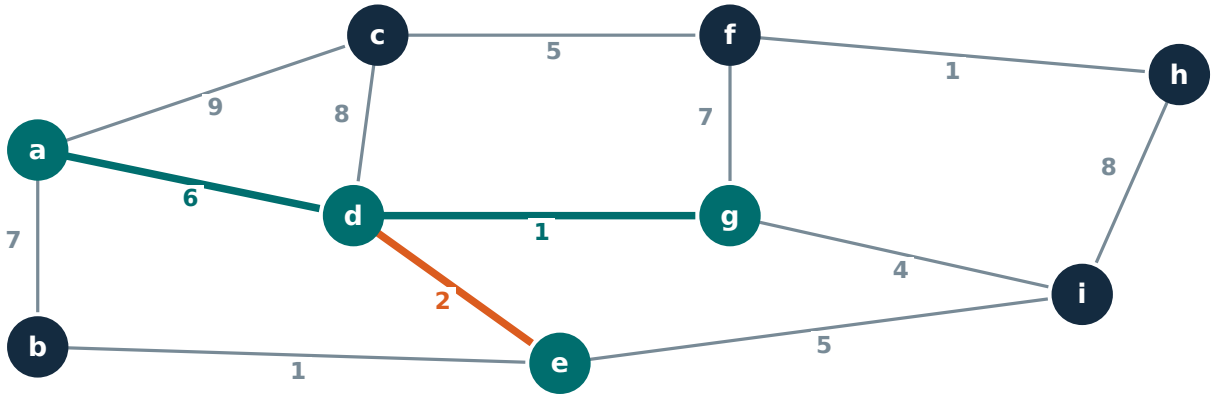


S={a,d}. d—g=1 é a menor aresta com uma ponta dentro e outra fora. Inclua g. b—e não serve ainda: ambas as pontas estão fora.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

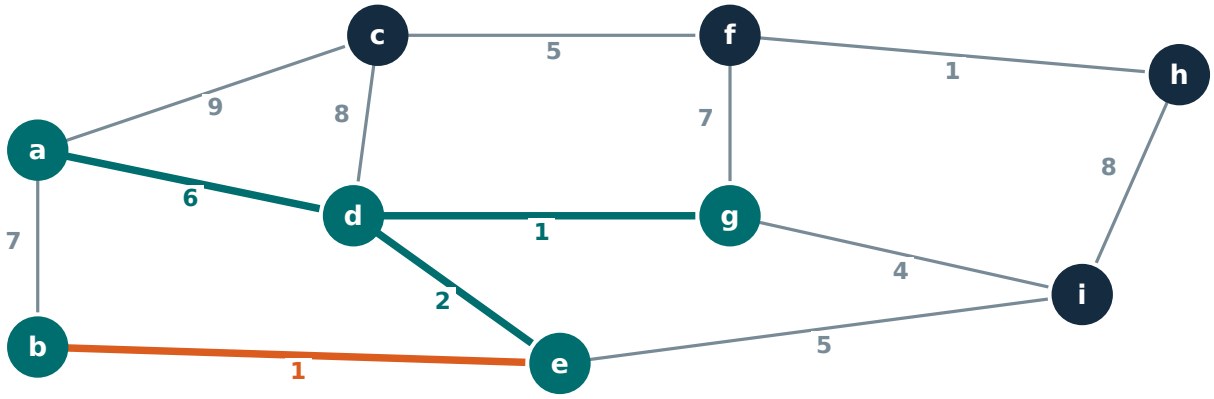
Execução visual • passos 3 e 4

3. Escolha d—e, peso 2 | total 9



Com g incluído, a menor saída é d—e=2. d—g já tem ambas as pontas dentro e não entra de novo.

4. Escolha b—e, peso 1 | total 10

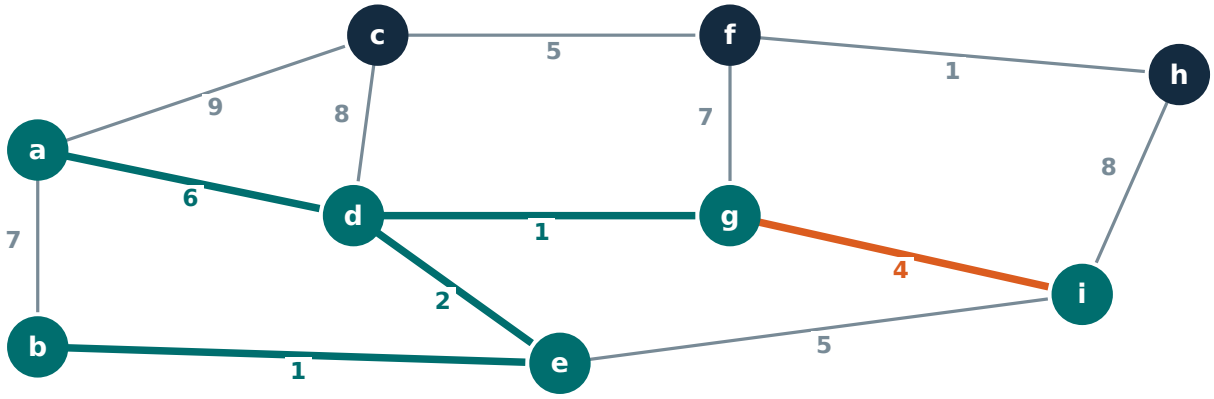


Agora e pertence à árvore. b—e=1 passa a cruzar a fronteira. Inclua b. O peso escolhido pode diminuir de uma etapa para outra.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

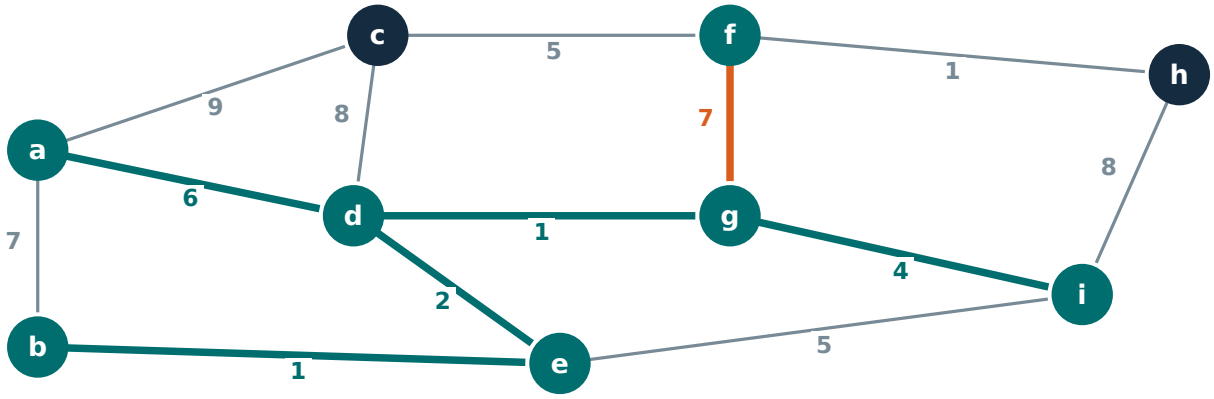
Execução visual • passos 5 e 6

5. Escolha g—i, peso 4 | total 14



Faltam c, f, h e i. g—i=4 é a menor ligação entre a árvore e esses vértices. Inclua i.

6. Escolha f—g, peso 7 | total 21

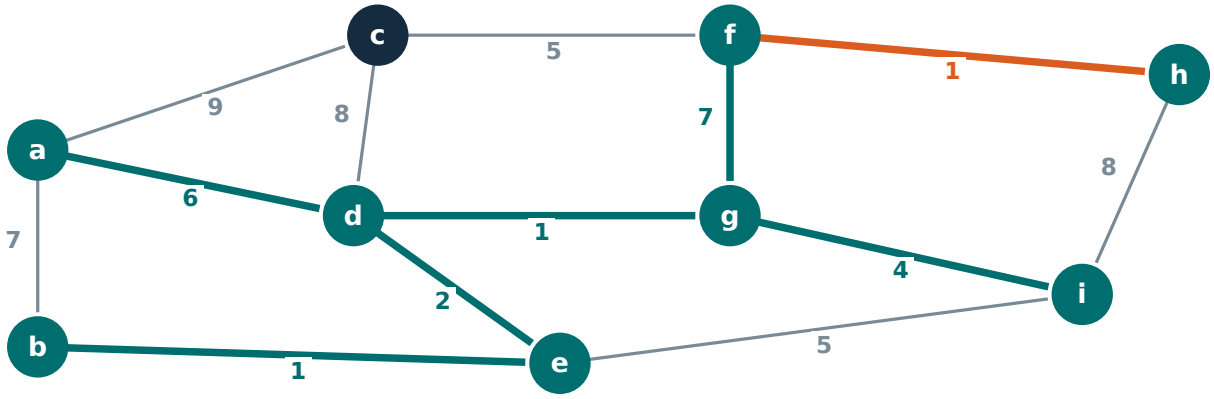


Faltam c, f e h. f—g=7 é a menor saída. c—f=5 e f—h=1 ainda têm ambas as pontas fora. Inclua f.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

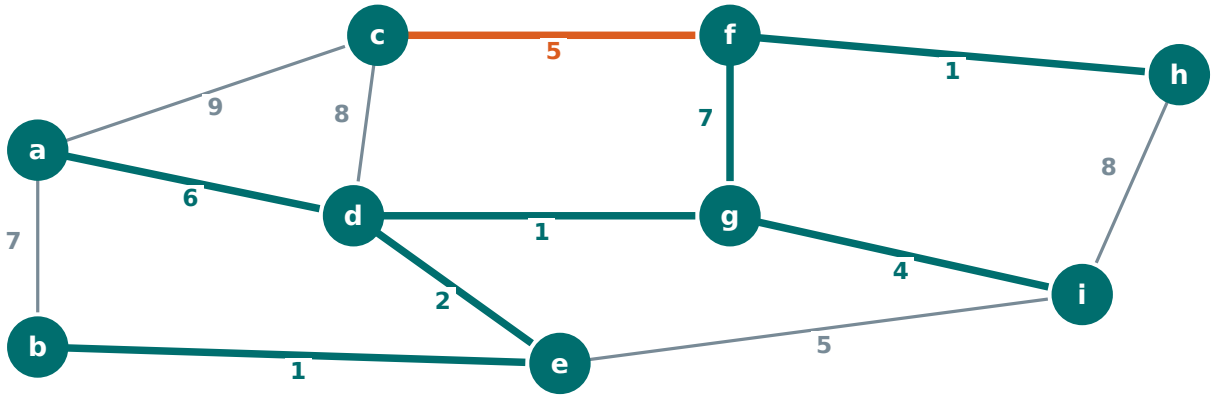
Execução visual • passos 7 e 8

7. Escolha f—h, peso 1 | total 22



Com f selecionado, f—h=1 vira candidata e é a menor saída. Inclua h.

8. Escolha c—f, peso 5 | total 27



Só c está fora. Compare c—f=5, c—d=8 e c—a=9. Inclua c—f. Todos estão conectados; pare.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

Por que a escolha é segura?

A fronteira entre S e V−S é um corte. Para alcançar os vértices de fora, qualquer árvore geradora precisa cruzá-lo. Uma aresta de menor peso que cruza esse corte pode pertencer a uma AGM.

Intuição da troca: se uma AGM não usa a aresta escolhida, adicione-a. Isso cria um ciclo. Nele há outra aresta cruzando o mesmo corte; remova essa outra. Como a escolhida era mínima no corte, o custo não aumenta.

• A saída inclui todos os vértices, exatamente n−1 arestas e nenhum ciclo.

• A soma 27 é o custo da árvore inteira. Não significa que a distância de a a cada vértice seja mínima nessa árvore.

• Empates podem produzir AGMs diferentes de mesmo custo. Pesos todos distintos garantem unicidade; a recíproca não vale.

• Implementação simples: $O(n^2)$. Com listas e heap binário: $O((n+m) \log n)$, usualmente $O(m \log n)$ no grafo conexo.

> **Erro frequente**
>
> Em Prim, procure a menor aresta que liga a árvore atual a um vértice novo. A menor aresta do grafo inteiro pode não ser elegível.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

Kruskal: unir componentes sem ciclos

O objetivo e as condições são os mesmos de Prim. Kruskal começa com todos os vértices isolados e examina as arestas globalmente, da menor para a maior.

> **A regra dos slides**
>
> Aceite a próxima aresta de menor peso somente se não formar ciclo com as já aceitas. Equivale a exigir que suas pontas estejam em componentes diferentes.

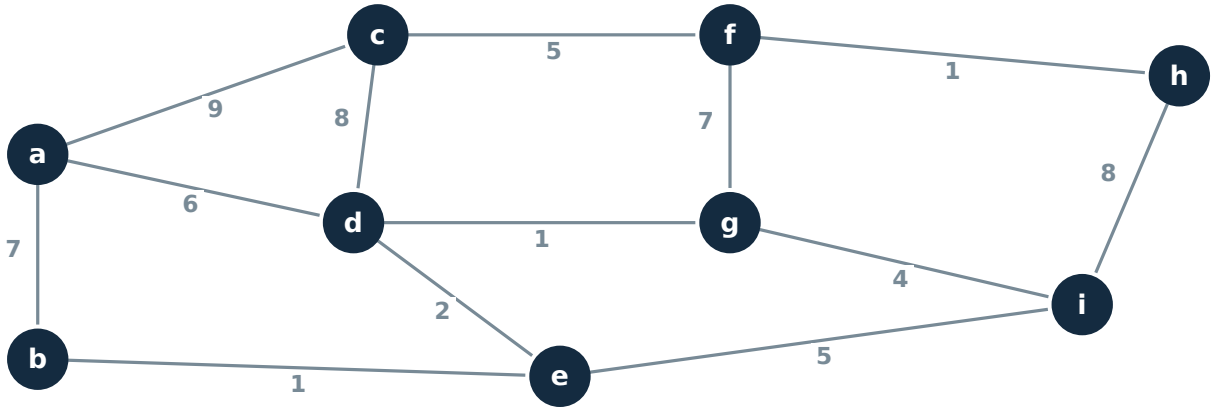
```
ordene as arestas por peso não decrescente
T = conjunto vazio
crie um componente para cada vértice
para cada aresta (u,v), nessa ordem:
    se componente(u) != componente(v):
        acrescente (u,v) a T
        una os dois componentes
    se |T| == n−1: pare
retorne T
```

O slide inicializa T com a primeira aresta; aqui usamos T vazio e deixamos o laço aceitá-la. É a mesma construção no exemplo sem laços. Union-Find permite consultar e unir componentes com eficiência.

Complexidade O(m log m), dominada pela ordenação. Em grafo desconexo, processe as arestas disponíveis: a saída será uma floresta geradora mínima.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

A ordem das arestas muda a construção



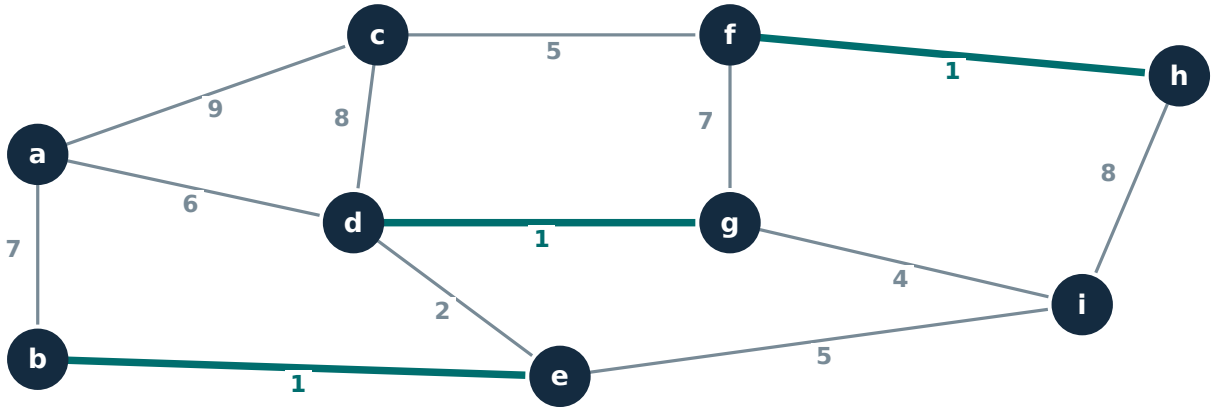
Ordem	Aresta / peso	Decisão	Total
1	b—e / 1	ACEITA	1
2	d—g / 1	ACEITA	2
3	f—h / 1	ACEITA	3
4	d—e / 2	ACEITA	5
5	g—i / 4	ACEITA	9
6	c—f / 5	ACEITA	14
7	e—i / 5	REJEITA: ciclo	14
8	a—d / 6	ACEITA	20
9	a—b / 7	REJEITA: ciclo	20
10	f—g / 7	ACEITA	27

Empates: ordem alfabética dos extremos. Paramos em f—g ao atingir 8 arestas aceitas. Não é necessário examinar as arestas posteriores.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

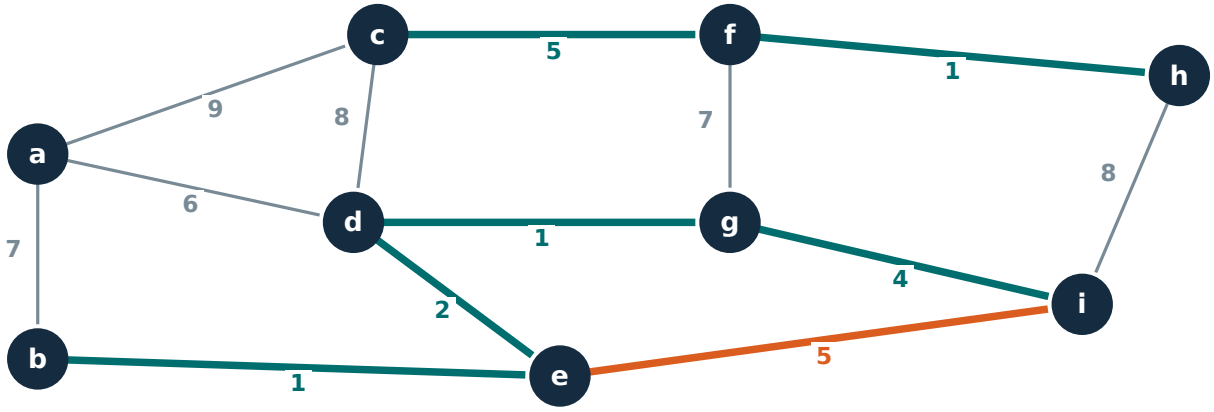
Uma floresta cresce; um ciclo é recusado

f—h: aceita | total 3



As três arestas de peso 1 unem b a e, d a g e f a h. A floresta tem várias partes: isso é esperado em Kruskal.

e—i: rejeita | total 14

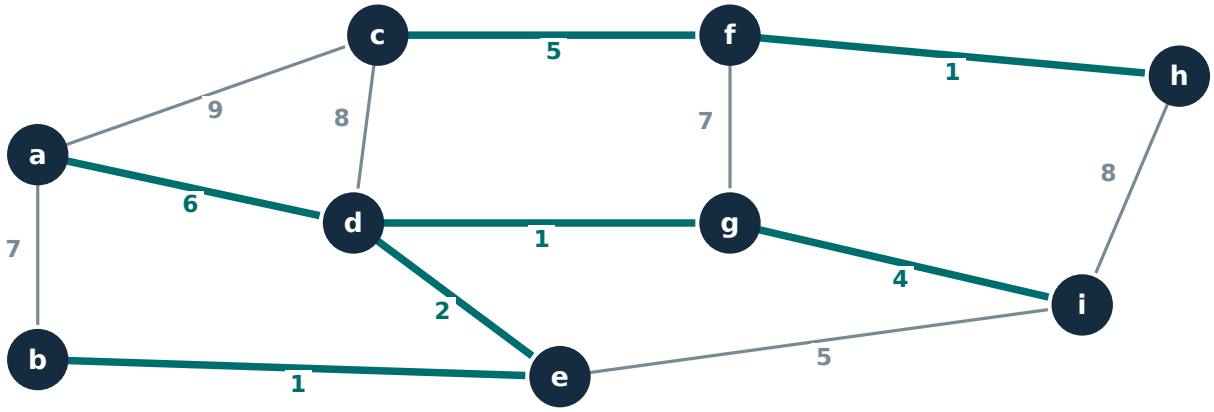


Já existe o caminho e—d—g—i. Acrescentar e—i=5 fecharia o ciclo e—d—g—i—e. Rejeite.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

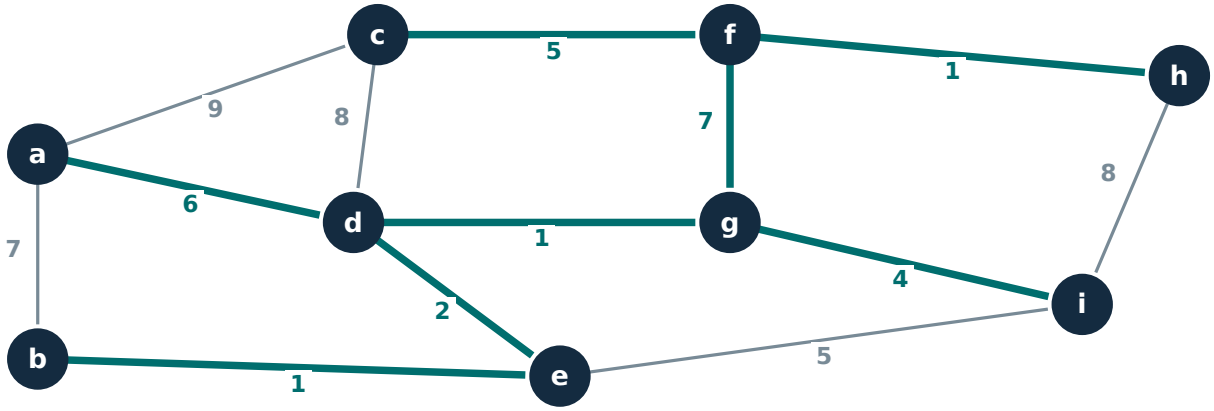
As últimas uniões e a árvore completa

a—d: aceita | total 20



a—d=6 conecta a à componente maior. Restam duas componentes: {a,b,d,e,g,i} e {c,f,h}.

f—g: aceita | total 27



a—b=7 foi recusada por fechar ciclo. f—g=7 une as duas componentes restantes. Custo final 27, igual ao de Prim.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

Prim × Kruskal: mesma meta, outra decisão

Aspecto	Prim	Kruskal
Começo	Uma raiz.	Todos os vértices isolados.
Candidatas	Arestas saindo da árvore atual.	Todas, ordenadas por peso.
Escolha	Menor aresta cruzando a fronteira.	Menor aresta unindo componentes.
Estado parcial	Uma árvore.	Uma floresta que vai se unindo.
Evita ciclo	Inclui um vértice novo.	Recusa extremos já conectados.
Custo do exemplo	27.	27.

Kruskal também usa a propriedade do corte: a menor aresta elegível que liga componentes é segura. Verificar que a saída é uma árvore não prova que ela é mínima; a regra de escolha segura é que garante o ótimo.

> **Para responder na prova**
>
> Mostre a ordem de análise, marque aceita/rejeita, justifique cada rejeição por ciclo e some somente as arestas aceitas. Em Prim, mostre o conjunto de vértices selecionados.

Zenilton, slides 12, pp. PDF 9–11 e 39–40; West, §2.3, pp. 95–97.

Dijkstra: fechar a menor distância provisória

Encontra caminhos mínimos de uma origem s aos vértices alcançáveis. Funciona em grafos direcionados ou não direcionados com pesos não negativos; zero é permitido.

> **A regra dos slides**
>
> Escolha o vértice não finalizado com menor d[v], finalize-o e relaxe suas arestas de saída. Atualize o predecessor quando uma distância melhorar.

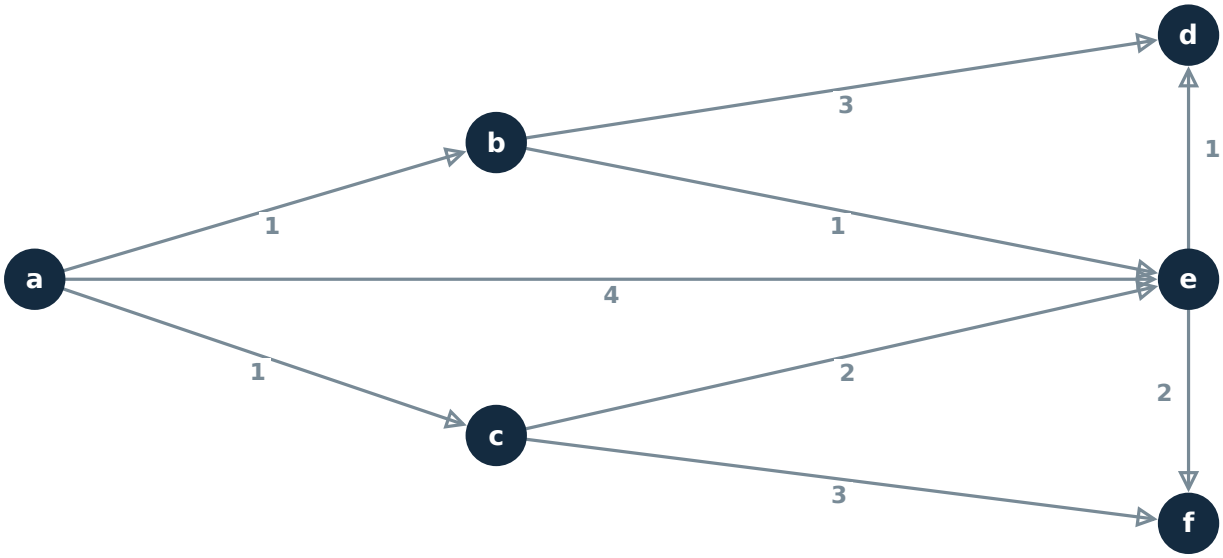
```
para cada v: d[v] = ∞; pred[v] = vazio
d[s] = 0; S = conjunto vazio
enquanto houver vértice fora de S com d finita:
    u = vértice fora de S com menor d[u]
    inclua u em S
    para cada aresta de saída (u,v):
        se d[v] > d[u] + w(u,v):
            d[v] = d[u] + w(u,v)
            pred[v] = u
```

Finalizar é afirmar que aquela distância não vai melhorar. Com pesos não negativos, continuar um caminho não diminui seu custo. Se não há distância finita fora de S, os demais vértices são inalcançáveis.

Custo $O(n^2)$ com seleção linear; $O((n+m) \log n)$ com heap binário e listas de adjacência.

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

O exemplo da aula: origem a



Grafo do slide 10. As setas verticais são e→d e e→f.

Finalizado	a	b	c	d	e	f
inicial	0	∞	∞	∞	∞	∞
a	0	1	1	∞	4	∞
b	0	1	1	4	2	∞
c	0	1	1	4	2	4
e	0	1	1	3	2	4
d	0	1	1	3	2	4
f	0	1	1	3	2	4

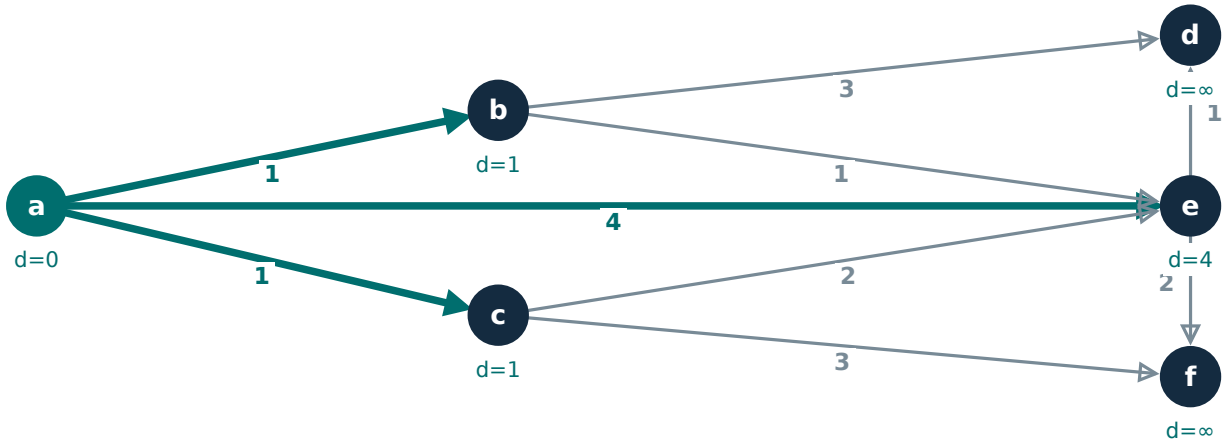
Compare o caminho inteiro

a→e custa 4, mas a→b→e custa 1+1=2. Dijkstra compara o custo acumulado, não apenas o peso da última aresta.

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

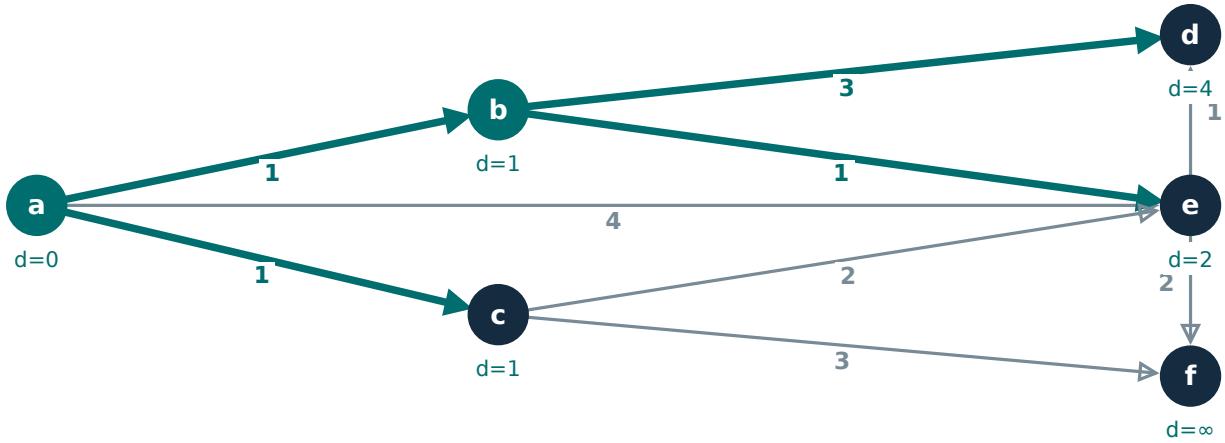
Execução visual • etapas 1 e 2

1. Finalize a, distância 0



Melhoras: b: ∞ → 1 (via a); c: ∞ → 1 (via a); e: ∞ → 4 (via a).

2. Finalize b, distância 1

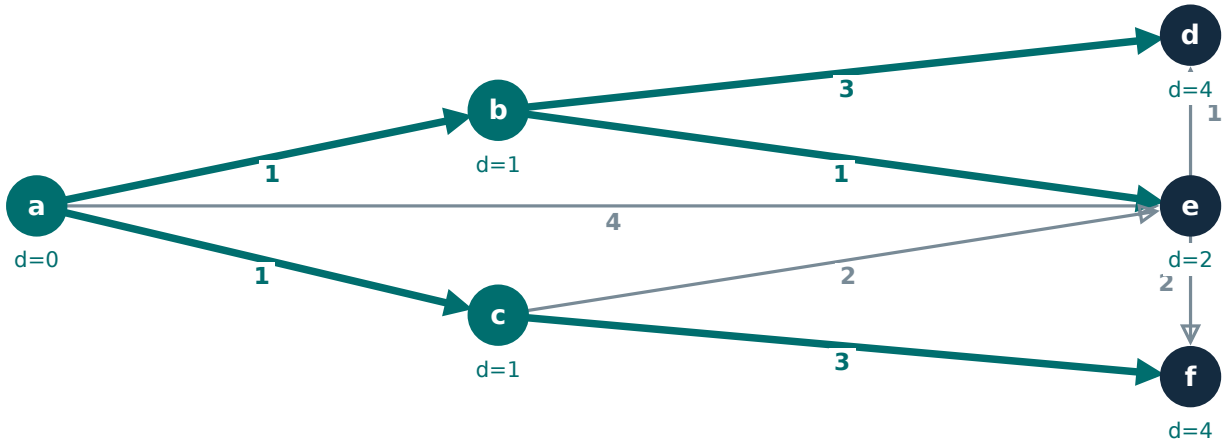


Melhoras: d: ∞ → 4 (via b); e: 4 → 2 (via b).

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

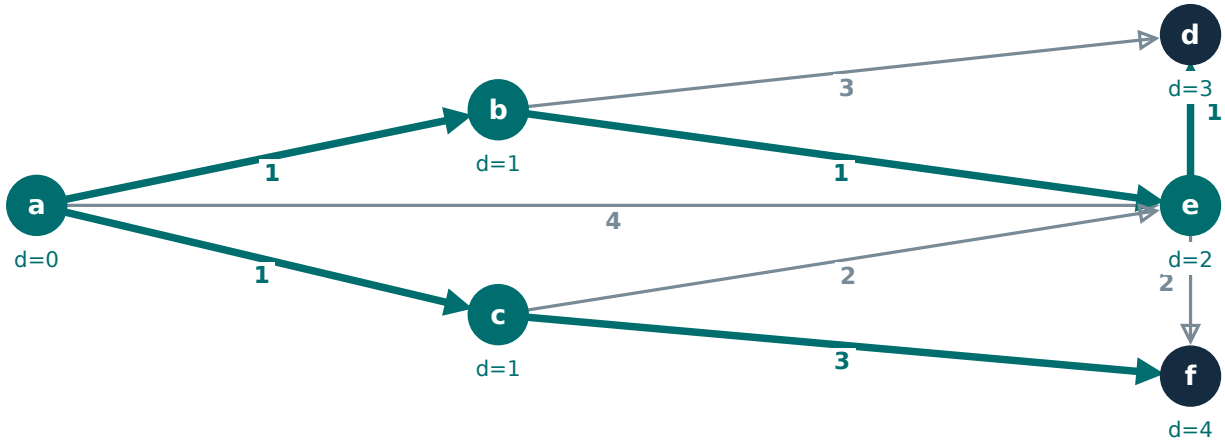
Execução visual • etapas 3 e 4

3. Finalize c, distância 1



Melhoras: f: ∞ → 4 (via c).

4. Finalize e, distância 2

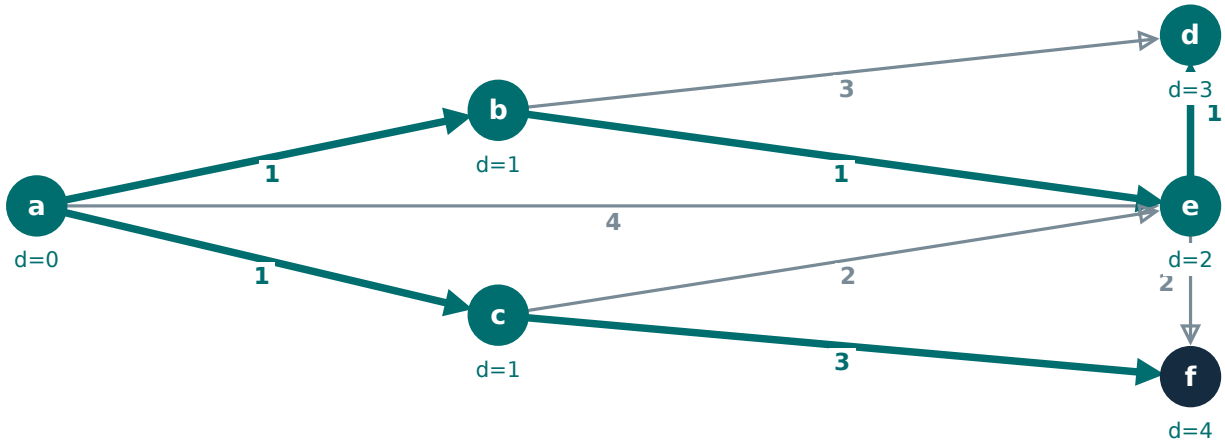


Melhoras: d: 4 → 3 (via e).

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

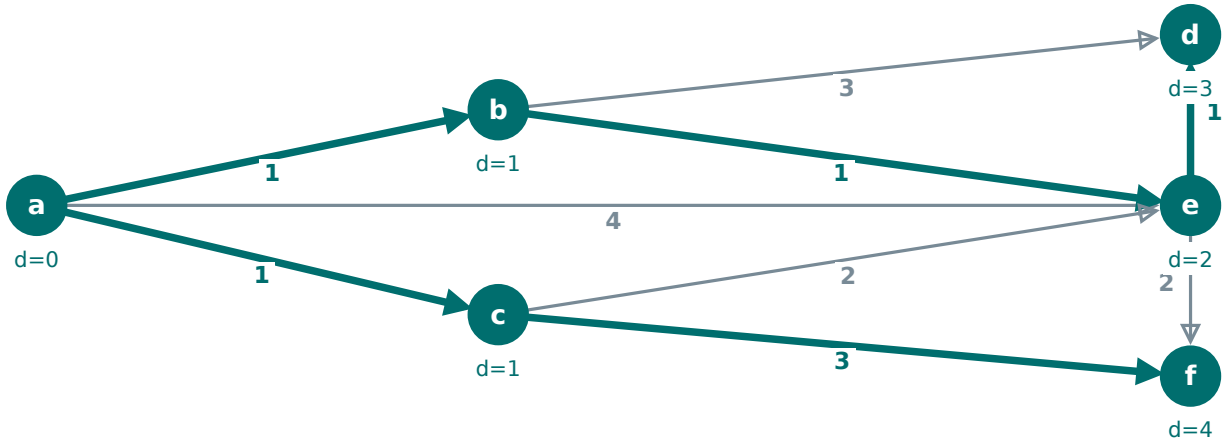
Execução visual • etapas 5 e 6

5. Finalize d, distância 3



Melhoras: nenhuma. As estimativas existentes já são tão boas quanto os candidatos.

6. Finalize f, distância 4



Melhoras: nenhuma. As estimativas existentes já são tão boas quanto os candidatos.

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

Reconstruir os caminhos pelos predecessores

Destino	Distância	Caminho
a	0	a
b	1	a→b
c	1	a→c
d	3	a→b→e→d
e	2	a→b→e
f	4	a→c→f

Para d, siga os predecessores ao contrário: d←e←b←a. Inverta essa sequência. Para f, a→b→e→f também custa 4; como não trocamos predecessor em empate, mantivemos o caminho via c.

> **Prim × Dijkstra**
>
> Prim usa o peso da aresta de ligação à árvore. Dijkstra usa d[u]+w(u,v), a distância acumulada da origem. Uma árvore de caminhos mínimos não precisa ser uma AGM.

Por que pesos negativos quebram a regra?

Considere s→x=2, s→y=5 e y→x=−10. Dijkstra finalizaria x com 2 antes de y. Mas s→y→x custa −5. O algoritmo padrão não pode tratar x como definitivo: use Bellman-Ford.

Zenilton, slides 13, pp. PDF 7–10 e 24–25; West, §2.3, pp. 97–99.

Bellman-Ford: repetir as relaxações

Resolve caminhos mínimos a partir de uma origem, inclusive com arestas negativas. O problema surge quando um ciclo de soma negativa é alcançável a partir da origem.

A regra dos slides

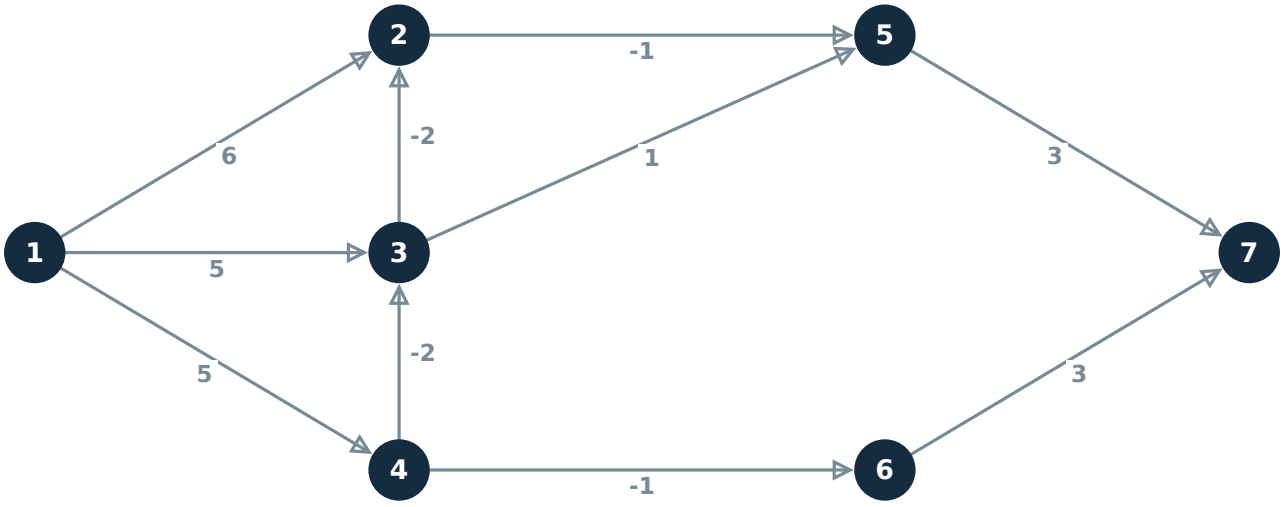
Inicialize d[s]=0 e demais distâncias com ∞. Varra todas as arestas até n−1 vezes. Se uma varredura inteira não mudar nada, pare antes.

```
d[s] = 0; demais d = ∞; pred = vazio
repita no máximo n−1 vezes:
    mudou = falso
    para cada aresta (u,v):
        se d[u] != ∞ e d[v] > d[u] + w(u,v):
            d[v] = d[u] + w(u,v); pred[v] = u
            mudou = verdadeiro
    se não mudou: interrompa
para cada (u,v) com d[u] finita:
    se d[v] > d[u] + w(u,v):
        sinalize ciclo negativo alcançável
```

Por que n−1? Sem ciclos negativos relevantes, existe um caminho mínimo simples para cada destino alcançável: no máximo n vértices e n−1 arestas. Custo O(nm), memória auxiliar O(n).

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

O grafo da aula tem pesos negativos



Exemplo do slide 17; origem 1.

Ordem fixa de varredura neste guia: 1→2, 1→3, 1→4, 2→5, 3→2, 3→5, 4→3, 4→6, 5→7, 6→7.

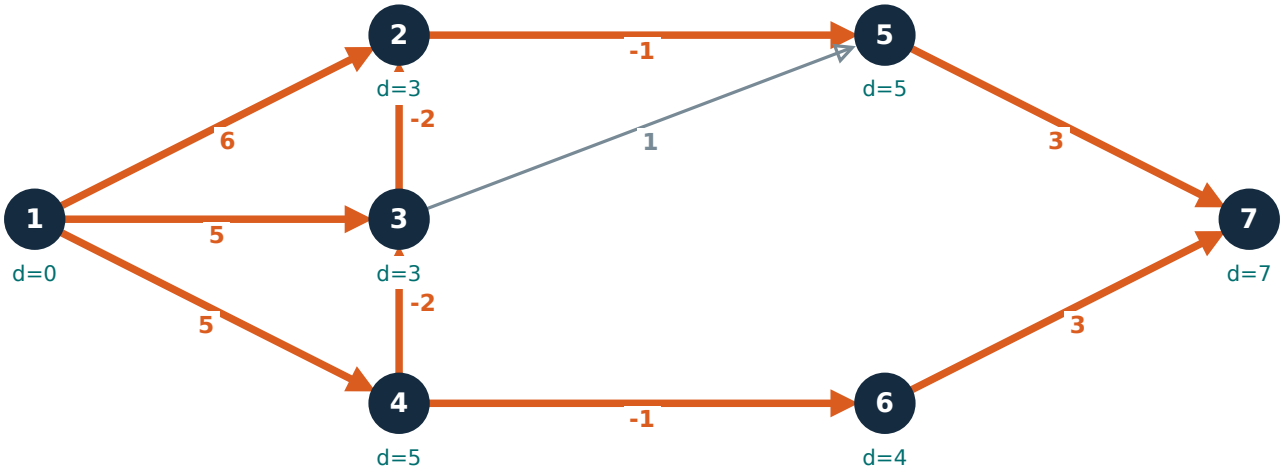
Estado	1	2	3	4	5	6	7
inicial	0	∞	∞	∞	∞	∞	∞
varredura 1	0	3	3	5	5	4	7
varredura 2	0	1	3	5	2	4	5
varredura 3	0	1	3	5	0	4	3
varredura 4	0	1	3	5	0	4	3

Atualização no próprio vetor

Como no pseudocódigo dos slides, uma melhora pode ser usada na mesma varredura. A ordem das arestas muda os estados intermediários, mas preserva o resultado final válido.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Varredura 1: o que mudou?

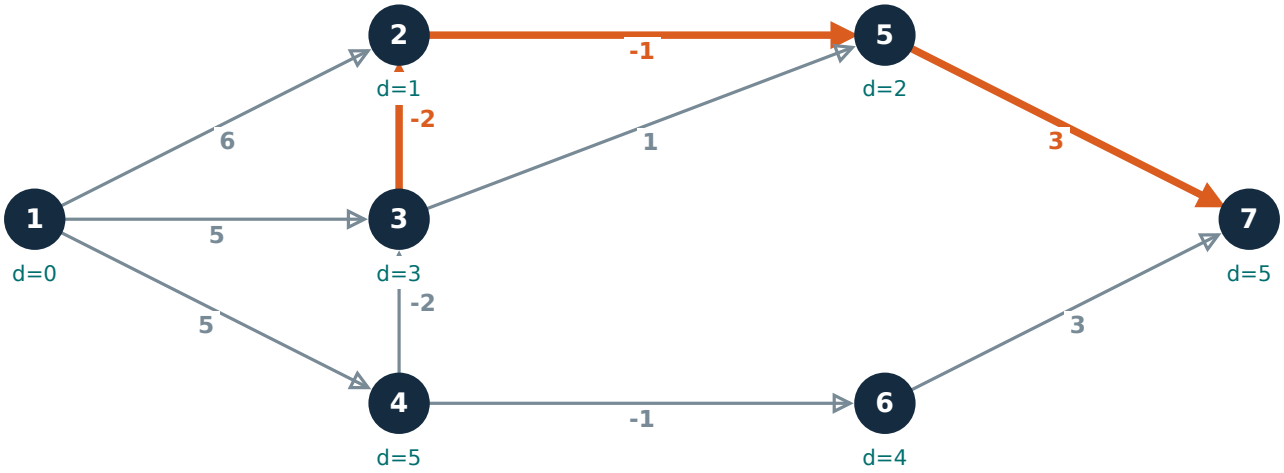


Aresta	Conta na hora da relaxação	Antes → depois
1→2 (6)	0 + (6) = 6	∞ → 6
1→3 (5)	0 + (5) = 5	∞ → 5
1→4 (5)	0 + (5) = 5	∞ → 5
2→5 (-1)	6 + (-1) = 5	∞ → 5
3→2 (-2)	5 + (-2) = 3	6 → 3
4→3 (-2)	5 + (-2) = 3	5 → 3
4→6 (-1)	5 + (-1) = 4	∞ → 4
5→7 (3)	5 + (3) = 8	∞ → 8
6→7 (3)	4 + (3) = 7	8 → 7

A tabela mostra apenas as melhorias. Todas as outras arestas também foram testadas. Se o custo de um predecessor diminuir depois de sua saída já ter sido examinada, a melhora pode chegar ao sucessor na próxima varredura.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Varredura 2: o que mudou?

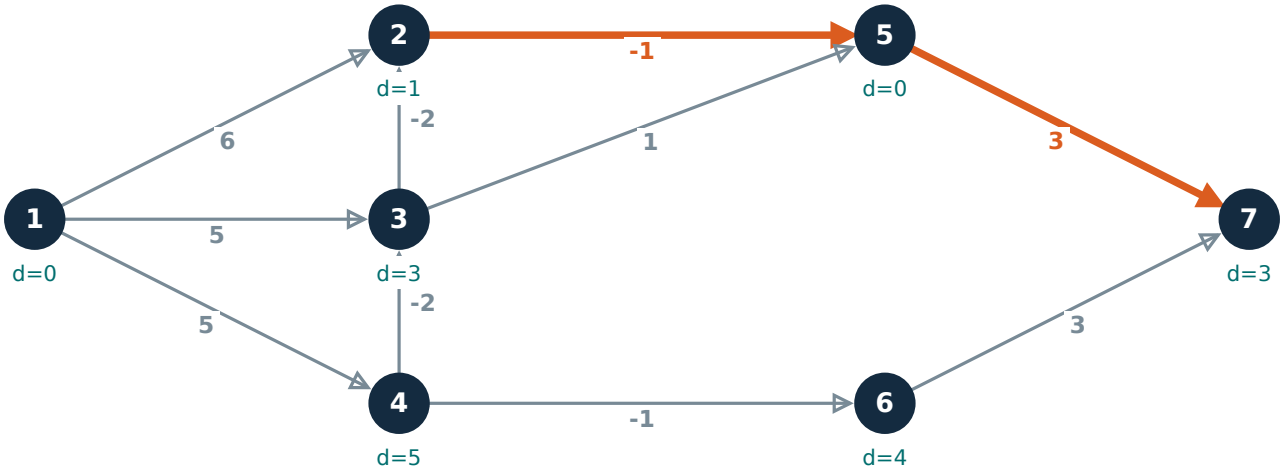


Aresta	Conta na hora da relaxação	Antes → depois
2→5 (-1)	3 + (-1) = 2	5 → 2
3→2 (-2)	3 + (-2) = 1	3 → 1
5→7 (3)	2 + (3) = 5	7 → 5

A tabela mostra apenas as melhorias. Todas as outras arestas também foram testadas. Se o custo de um predecessor diminuir depois de sua saída já ter sido examinada, a melhora pode chegar ao sucessor na próxima varredura.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Varredura 3: o que mudou?

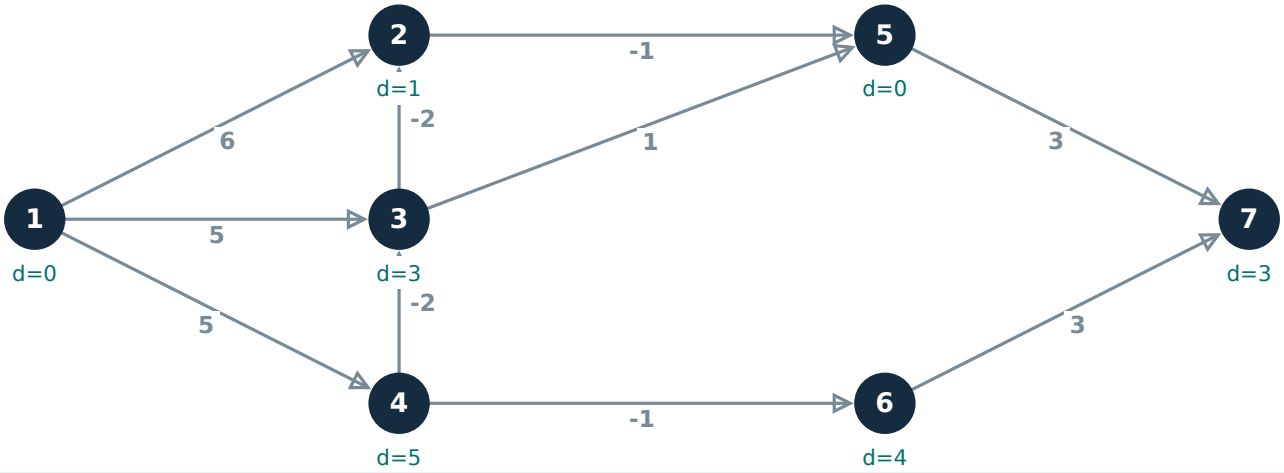


Aresta	Conta na hora da relaxação	Antes → depois
2→5 (-1)	1 + (-1) = 0	2 → 0
5→7 (3)	0 + (3) = 3	5 → 3

A tabela mostra apenas as melhorias. Todas as outras arestas também foram testadas. Se o custo de um predecessor diminuir depois de sua saída já ter sido examinada, a melhora pode chegar ao sucessor na próxima varredura.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Varredura 4: o que mudou?



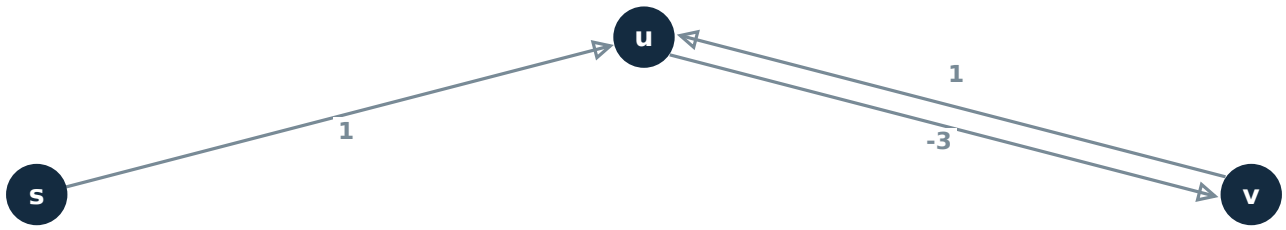
Convergência

Nenhuma aresta melhora uma distância. Repetir a mesma varredura não mudaria nada. Podemos terminar antes do limite de seis varreduras.

Resultado: d=[0,1,3,5,0,4,3]. O caminho até 7 é 1→4→3→2→5→7, com custo 5−2−2−1+3=3.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Ciclo negativo: quando não há mínimo



Exemplo complementar. O ciclo u→v→u custa −2.

Depois de chegar a u, cada volta reduz o custo em 2. Não existe um menor valor finito para u, v e os destinos alcançáveis a partir desse ciclo.

O teste correto

Após até n−1 varreduras, faça uma verificação adicional. Se ainda houver aresta relaxável cuja origem tenha distância finita, há ciclo negativo alcançável. Melhorar apenas na varredura n−1 pode ser normal.

Ciclo negativo numa região inalcançável de s não é detectado por essa execução e não invalida os caminhos a partir de s. Para afetar um destino t, o ciclo precisa ser alcançável de s e permitir chegar a t.

Precisão na leitura dos slides: o limite n−1 vale para caminhos simples; um passeio pode repetir vértices. A checagem extra torna precisa a observação sobre atualizações persistentes.

Zenilton, slides 14, pp. PDF 13–17. Referências completas no fim do guia.

Floyd-Warshall: permitir intermediários

Calcula distâncias entre todos os pares. Aceita arestas negativas; um par só tem mínimo finito quando nenhum ciclo negativo pode ser utilizado em um percurso entre sua origem e seu destino.

> **A regra dos slides**
>
> Na rodada k, teste se ir de i até k e de k até j melhora a rota de i até j: D[i,j] = min(D[i,j], D[i,k]+D[k,j]).

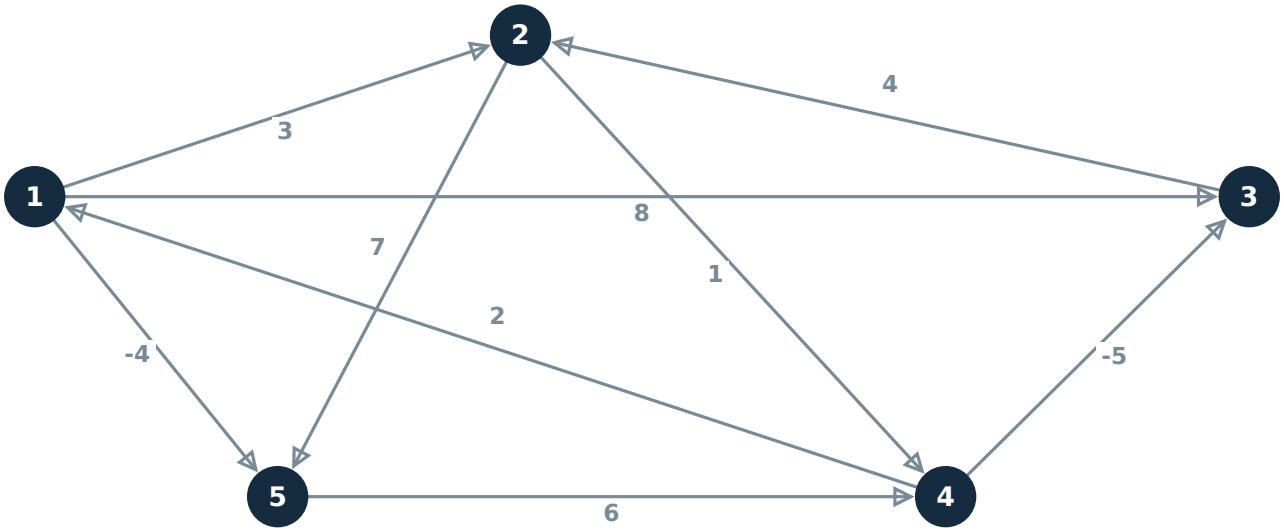
D^0 contém as arestas diretas, zero na diagonal e ∞ onde não há aresta. D^k permite os vértices 1,...,k como intermediários. Depois de k=n, todos estão permitidos.

```
inicialize D[i,i] = 0; demais D[i,j] = ∞
para cada aresta: D[i,j] = min(D[i,j], w(i,j))
para k de 1 até n:
    para i de 1 até n:
        para j de 1 até n:
            se D[i,k] e D[k,j] forem finitos:
                D[i,j] = min(D[i,j], D[i,k]+D[k,j])
```

O laço k fica por fora: ele controla os intermediários liberados. Custo $O(n^3)$, memória $O(n^2)$. Se D[v,v]<0 ao final, existe ciclo negativo; pares capazes de passar por ele não têm mínimo finito.

Zenilton, slides 15, pp. PDF 11–22. Referências completas no fim do guia.

O grafo e a matriz inicial dos slides



Grafo do slide 22. A linha é a origem i; a coluna é o destino j.

D^0 • sem intermediários

i / j	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	∞	-5	0	∞
5	∞	∞	∞	6	0

D[1,5]=−4 vem da aresta direta. D[3,1]=∞ indica ausência de aresta direta; uma rota usando intermediários ainda pode existir.

Zenilton, slides 15, pp. PDF 11–22. Referências completas no fim do guia.

Matrizes após liberar 1 e 2

D^1 • intermediários permitidos: 1 até 1

i / j	1	2	3	4	5
1	0	3	8	∞	-4
2	∞	0	∞	1	7
3	∞	4	0	∞	∞
4	2	5 *	-5	0	-2 *
5	∞	∞	∞	6	0

D[4,2] muda de ∞ para 5: via 1, o custo é 2+(3)=5. * marca todas as células alteradas na rodada.

D^2 • intermediários permitidos: 1 até 2

i / j	1	2	3	4	5
1	0	3	8	4 *	-4
2	∞	0	∞	1	7
3	∞	4	0	5 *	11 *
4	2	5	-5	0	-2
5	∞	∞	∞	6	0

D[1,4] muda de ∞ para 4: via 2, o custo é 3+(1)=4. * marca todas as células alteradas na rodada.

Zenilton, slides 15, pp. PDF 11–22. Referências completas no fim do guia.

Matrizes após liberar 3 e 4

D^3 • intermediários permitidos: 1 até 3

i / j	1	2	3	4	5
1	0	3	8	4	-4
2	∞	0	∞	1	7
3	∞	4	0	5	11
4	2	-1 *	-5	0	-2
5	∞	∞	∞	6	0

D[4,2] muda de 5 para -1: via 3, o custo é -5+(4)=-1. * marca todas as células alteradas na rodada.

D^4 • intermediários permitidos: 1 até 4

i / j	1	2	3	4	5
1	0	3	-1 *	4	-4
2	3 *	0	-4 *	1	-1 *
3	7 *	4	0	5	3 *
4	2	-1	-5	0	-2
5	8 *	5 *	1 *	6	0

D[1,3] muda de 8 para -1: via 4, o custo é 4+(-5)=-1. * marca todas as células alteradas na rodada.

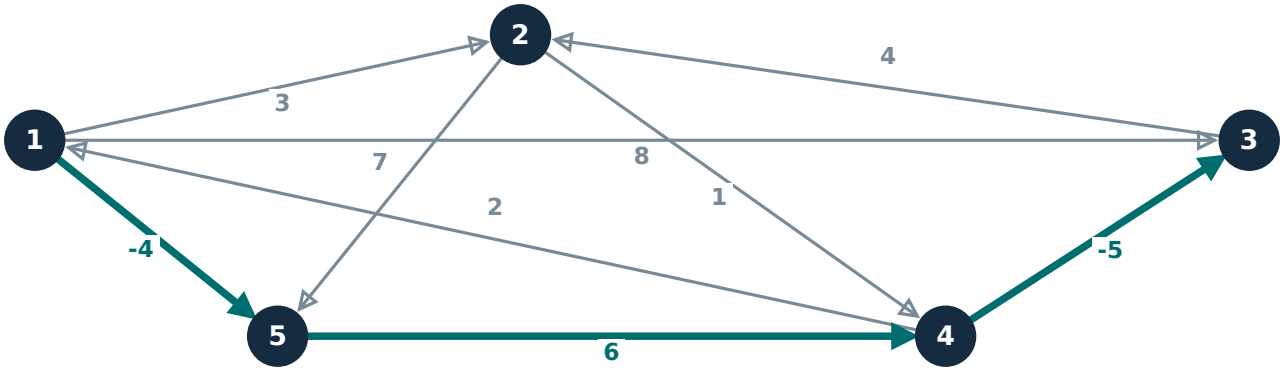
Zenilton, slides 15, pp. PDF 11–22. Referências completas no fim do guia.

Matriz final e caminho reconstruído

D^5 • todos os intermediários permitidos

i / j	1	2	3	4	5
1	0	1 *	-3 *	2 *	-4
2	3	0	-4	1	-1
3	7	4	0	5	3
4	2	-1	-5	0	-2
5	8	5	1	6	0

Ao liberar 5, D[1,3] melhora para D[1,5]+D[5,3]=−4+1=−3. A rota é 1→5→4→3, de custo −4+6−5=−3.



Para guardar os caminhos

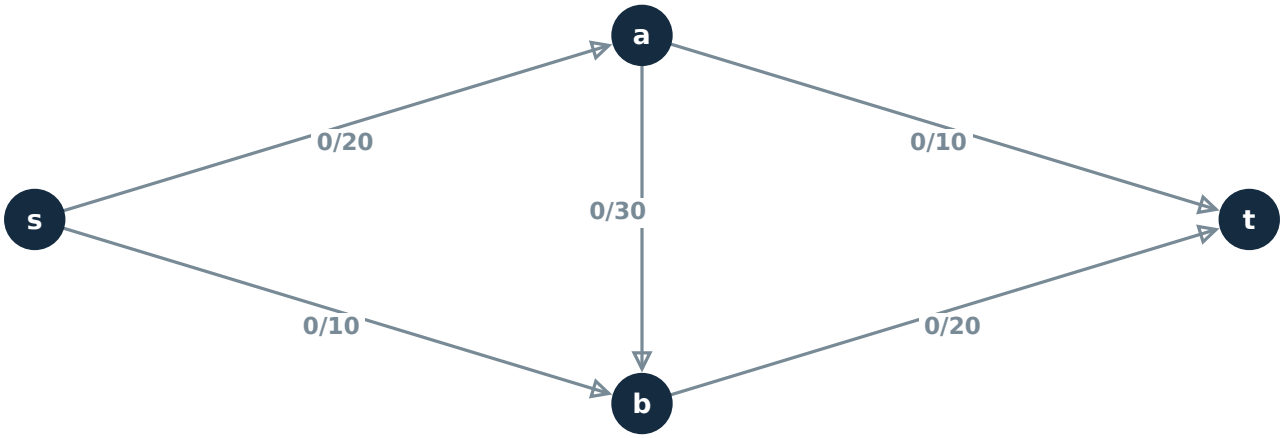
Use próximo[i,j]=j nas arestas diretas. Quando uma rota via k melhora D[i,j], copie próximo[i,j]=próximo[i,k]. Para reconstruir, avance de i para próximo[i,j] até chegar a j.

A diagonal final é zero: não há ciclo negativo neste exemplo. Distância negativa entre vértices distintos, sozinha, não indica ciclo negativo.

Zenilton, slides 15, pp. PDF 11–22. Referências completas no fim do guia.

Capacidade, conservação e valor do fluxo

Uma rede é um grafo direcionado com capacidades não negativas, fonte s e sumidouro t. Queremos enviar a maior quantidade possível de s para t, dividindo o envio por vários caminhos quando necessário.



Rede do slide 32. Cada rótulo é fluxo/capacidade: 0/20 significa fluxo zero e capacidade 20.

• Capacidade: 0 ≤ f(u,v) ≤ c(u,v). Não pode passar mais que a aresta suporta.

• Conservação: em cada vértice interno, total que entra = total que sai. Fonte e sumidouro são as exceções.

• Valor |f|: total que sai de s menos total que entra em s. Nesta rede nada entra em s; basta somar f(s,a)+f(s,b).

> **Fluxo não é apenas um caminho**
>
> Um caminho de maior gargalo ainda é apenas um caminho. Um fluxo combina caminhos e pode redistribuir envios anteriores. Por isso precisamos da rede residual.

Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Rede residual: espaço livre e possibilidade de voltar

Para cada aresta original u→v de capacidade c e fluxo f, a residual permite enviar mais c−f no sentido direto e cancelar até f no sentido reverso. Só desenhamos capacidades residuais positivas.



Original: foram enviados 7 de um máximo de 10.



Residual: cabem mais 3 adiante; até 7 podem ser cancelados no sentido inverso.

Caminho aumentante e gargalo

Um caminho de s até t na residual permite aumentar |f|. O aumento Δ é a menor capacidade residual do caminho. Nas arestas diretas some Δ; nas reversas subtraia Δ do fluxo original.

Reversa não significa mandar água fisicamente para trás. É o registro de que uma decisão anterior pode ser desfeita para abrir espaço a outra distribuição. Depois de aumentar, atualize a residual.

Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Ford-Fulkerson: aumentar enquanto for possível

Comece com fluxo zero e procure algum caminho aumentante. A escolha é livre; diferentes caminhos podem exigir quantidades muito diferentes de etapas.

```
f = zero em todas as arestas
construa a residual R
enquanto existir caminho P de s até t em R:
    Δ = menor capacidade residual em P
    para cada arco de P:
        se direto: some Δ ao fluxo original
        se reverso: subtraia Δ do fluxo original
    atualize R
retorne f
```

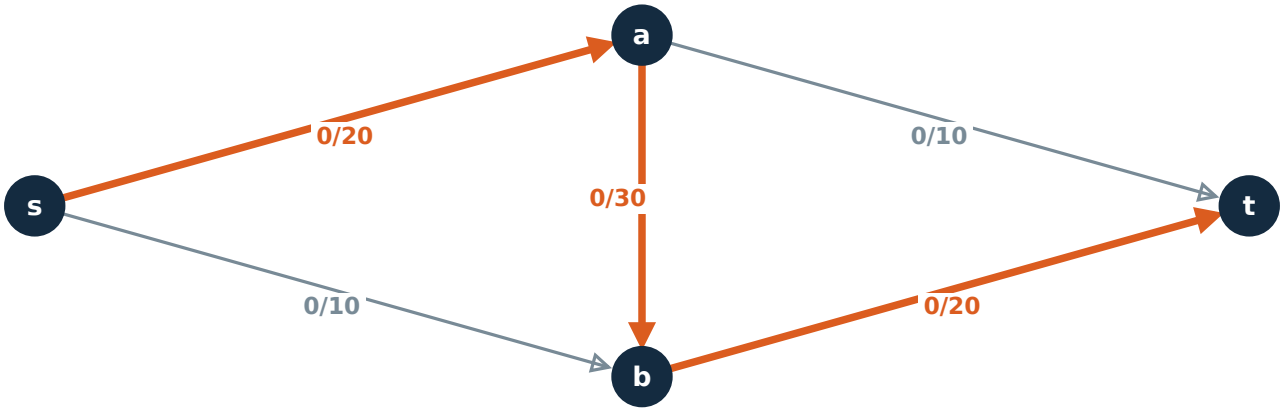
Condição de parada

Pare quando t não for alcançável de s na residual. Ausência de caminho aumentante significa fluxo máximo. Saturar um caminho específico não basta.

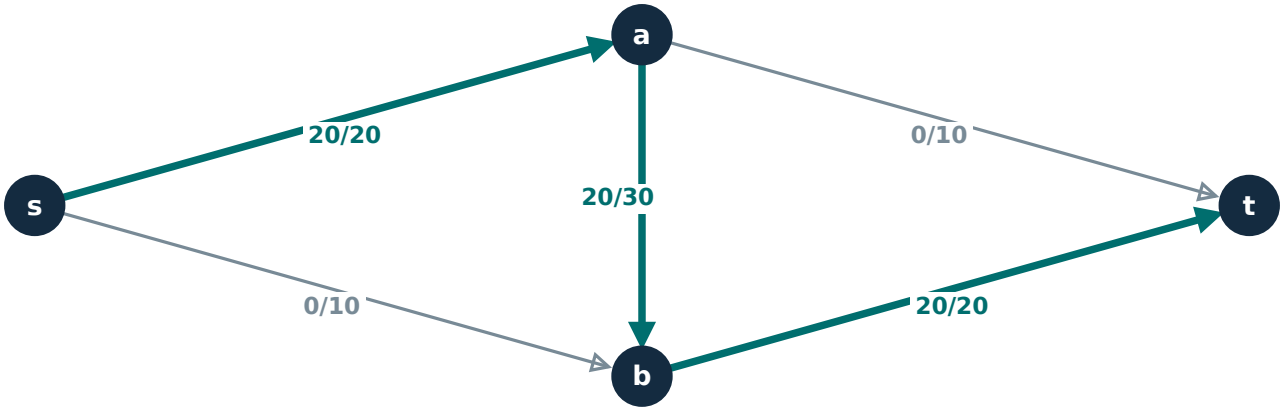
Com capacidades inteiras, cada aumento eleva o valor em pelo menos 1: O(mF), com F sendo o valor máximo. É pseudopolinomial, pois depende dos valores numéricos das capacidades. Com capacidades irracionais, escolhas arbitrárias podem não terminar.

Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Primeiro aumento: s → a → b → t



Capacidades residuais do caminho: 20, 30, 20. Gargalo Δ=min(20,30,20)=20. Some 20 às três arestas. O valor do fluxo passa a 20.

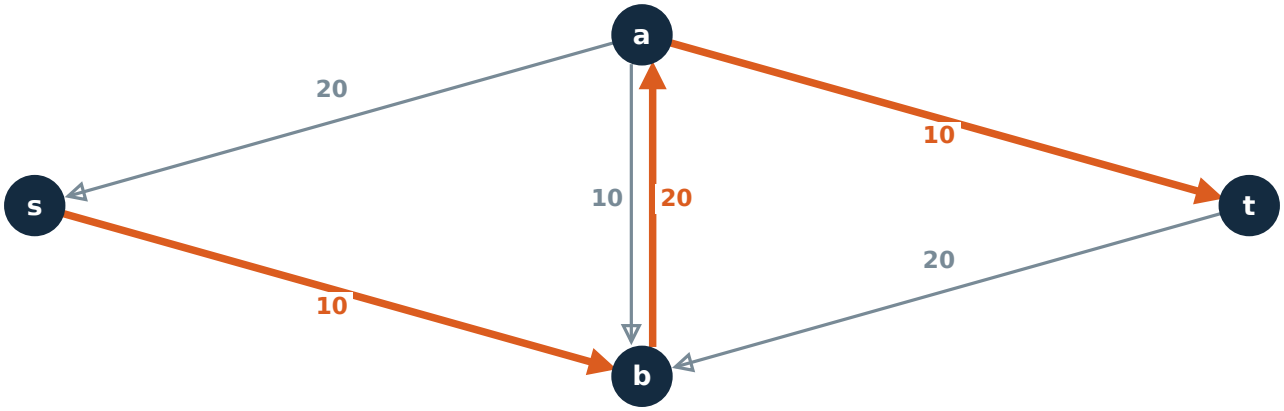


Por que ainda não acabou?

s→a e b→t saturaram, mas há espaço em s→b e a→t. Para usá-las juntas, precisaremos cancelar parte do envio anterior de a para b.

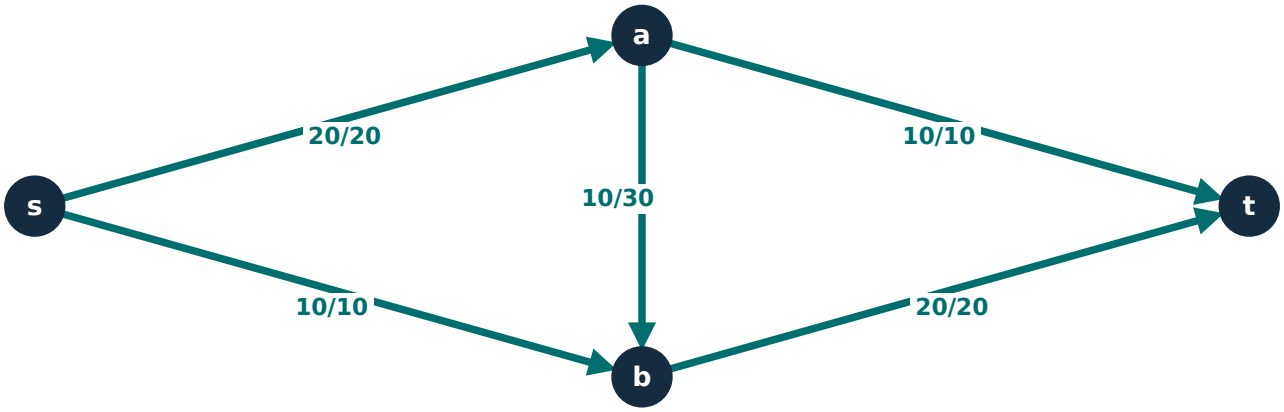
Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Segundo aumento: a reversa conserta o envio



Agora os rótulos são capacidades residuais. b→a pode cancelar envio de a→b.

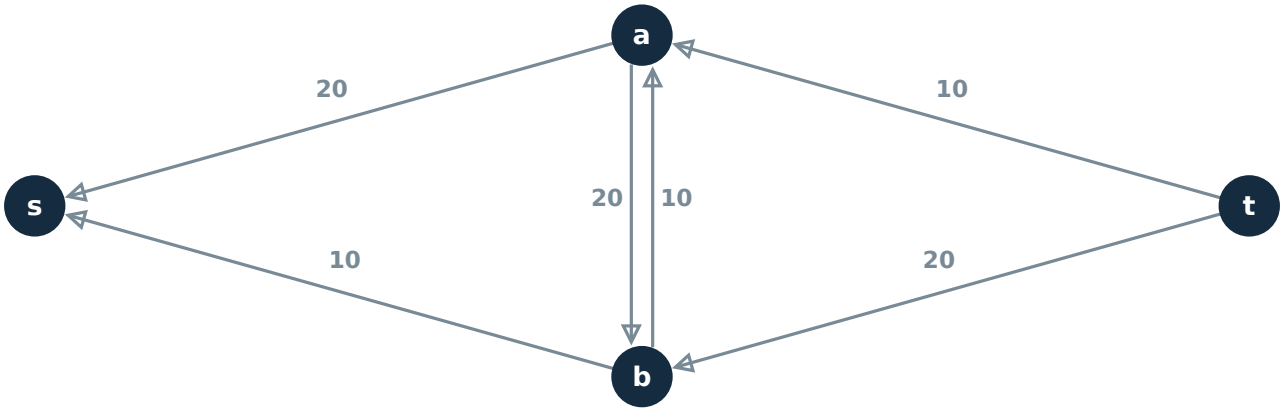
Caminho s→b→a→t: capacidades 10, 20, 10. Gargalo Δ=10. Some 10 a s→b e a→t; subtraia 10 de a→b.



Final: f(s,a)=20, f(s,b)=10, f(a,b)=10, f(a,t)=10, f(b,t)=20. Em a: entram 20 e saem 10+10. Em b: entram 10+10 e saem 20. Valor total=30.

Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Como provar que 30 é o máximo?



Residual final: nenhuma aresta sai de s; t não é alcançável.

S={s} é o conjunto alcançável da fonte na residual. O corte (S,V−S) tem capacidade c(s,a)+c(s,b)=20+10=30. Nenhum fluxo pode exceder a capacidade desse corte.

Certificado de ótimo

Há um fluxo viável de valor 30 e um corte de capacidade 30. Como todo fluxo é limitado por qualquer corte, o fluxo é máximo e esse corte é mínimo. É o teorema fluxo máximo = corte mínimo.

Em um corte direcionado, some as capacidades originais que saem de S para V−S. Não some automaticamente os dois sentidos. O fluxo líquido através do corte é o que sai menos o que entra.

Na prova: mostre fluxo/capacidade na rede original, caminho residual, gargalo, atualizações e ausência de caminho aumentante ou um corte de mesmo valor.

Zenilton, slides 16, pp. PDF 8, 14, 29–45; West, §4.3, pp. 176–180.

Edmonds-Karp: o caminho com menos arestas

É uma escolha sistemática dentro de Ford-Fulkerson: em cada residual, use BFS para achar um caminho aumentante com o menor número de arestas. As capacidades não são os comprimentos da BFS.

> **A regra dos slides**
>
> BFS na residual → caminho com menos arcos → gargalo → aumento e atualização. Faça uma nova BFS depois de cada aumento.

```
f = zero
enquanto uma BFS residual alcançar t:
    reconstrua P pelos pais da BFS
    Δ = mínimo das capacidades residuais de P
    aumente f ao longo de P
    atualize a residual
retorne f
```

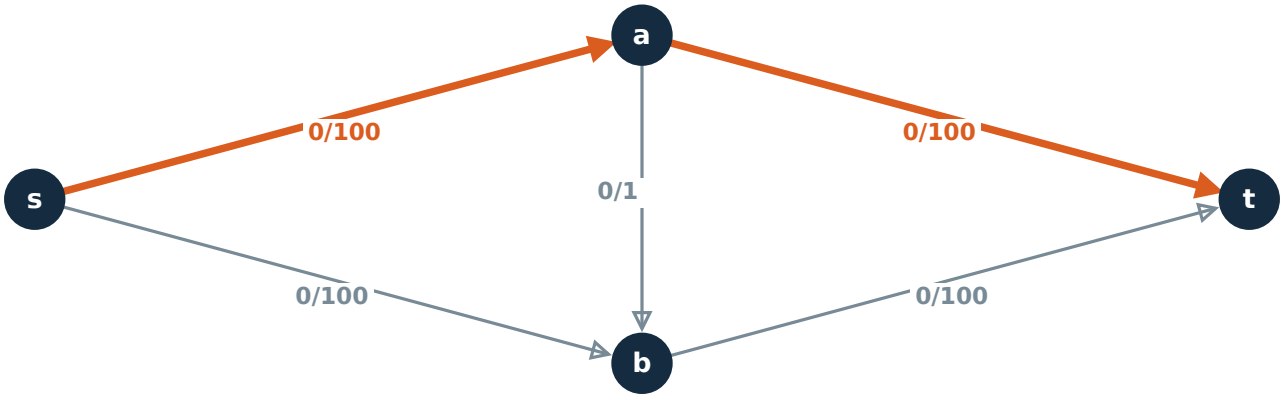
A BFS custa O(n+m), usualmente O(m) na parte relevante. Há O(nm) aumentos, total $O(nm^2)$. A garantia independe da magnitude numérica de F.

O arco a→b de capacidade 1 pode fazer escolhas ruins de Ford-Fulkerson aumentarem só 1 por vez. Edmonds-Karp prefere s→a→t ou s→b→t, com duas arestas, antes de s→a→b→t, com três.

Zenilton, slides 17, pp. PDF 20–36 e 62–63.

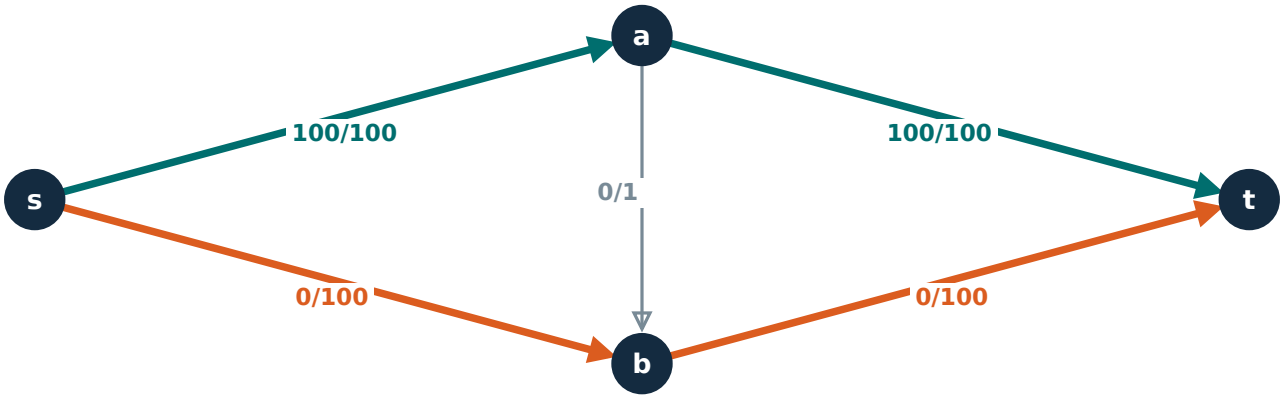
Exemplo da aula: duas BFS e fluxo 200

1. BFS escolhe s → a → t



Níveis: s=0; a=b=1; t=2. Visitando a antes de b, o pai de t é a. Gargalo=min(100,100)=100. Aumente em 100.

2. Nova BFS escolhe s → b → t



Envie mais 100. As duas saídas de s saturaram. Uma terceira BFS não chega a t. Valor máximo=200, igual ao corte {s} de capacidade 100+100.

Zenilton, slides 17, pp. PDF 20–36 e 62–63.

O que a BFS faz, de verdade?

Ação na primeira BFS	Fila	Pais definidos
Insere s	[s]	pai[s]=vazio
Remove s, descobre a,b	[a,b]	pai[a]=s; pai[b]=s
Remove a, descobre t	[b,t]	pai[t]=a
Reconstrói ao chegar a t	Pode encerrar	t←a←s

Siga apenas arcos residuais de capacidade positiva. Marque o vértice ao inseri-lo na fila para não redefinir seus pais. Reconstrua o caminho voltando de t pelos pais; depois inverta a sequência.

> **Três cuidados**
>
> Menos arestas não significa maior capacidade. BFS usa a residual atual, incluindo reversas. Depois da BFS, ainda é preciso calcular gargalo e atualizar o fluxo.

Por que melhora a garantia? Os comprimentos mínimos dos caminhos aumentantes não diminuem. Uma aresta só volta a limitar um aumento após uma mudança suficiente de níveis. Isso limita as ocorrências críticas e produz O(nm) aumentos.

Zenilton, slides 17, pp. PDF 20–36 e 62–63.

Dinic: um fluxo de bloqueio por fase

Dinic também usa BFS na residual, mas trabalha por fases. Cada fase constrói uma rede de níveis e envia fluxo até bloquear todos os caminhos s→t dessa rede.

Rede de níveis

Guarde só arcos residuais positivos u→v com nível[v]=nível[u]+1. Toda travessia avança um nível. A rede de níveis é acíclica.

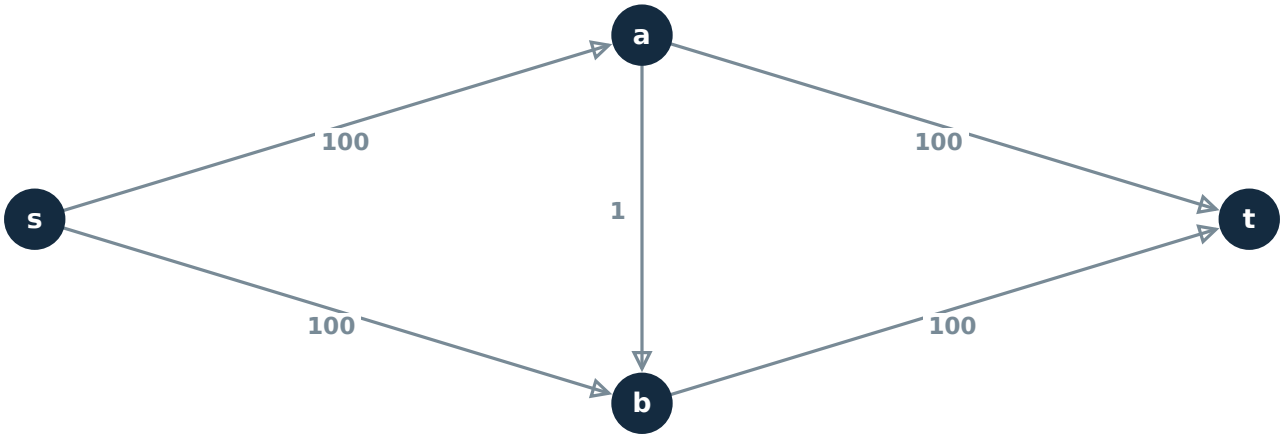
```
f = zero
enquanto BFS na residual alcançar t:
    monte os níveis e os arcos que avançam 1 nível
    zere os ponteiros de exploração desta fase
    enquanto DFS conseguir enviar fluxo até t:
        envie o gargalo e atualize as residuais
    # sem caminho nos níveis: fluxo de bloqueio
retorne f
```

A DFS respeita níveis e capacidades positivas. Ponteiros por vértice evitam reexaminar arcos inúteis. Fluxo de bloqueio não precisa ser máximo na rede de níveis; precisa bloquear todos os caminhos dessa rede.

Limite geral $O(n^2m)$: uma fase pode custar $O(nm)$, e há menos de n fases com t alcançável. Depois de um bloqueio, a nova distância BFS até t aumenta, se t ainda for alcançável.

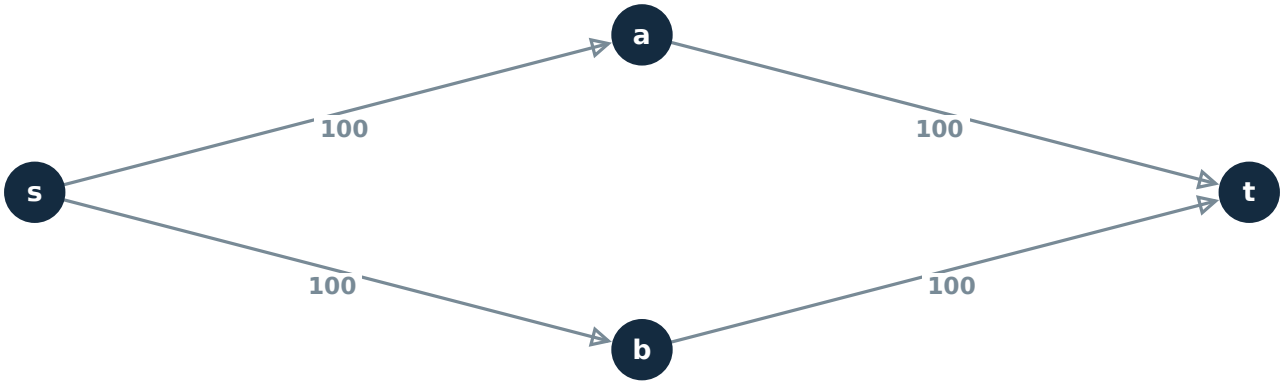
Zenilton, slides 17, pp. PDF 38–60 e 63.

Monte os níveis antes de mandar fluxo



A mesma rede de capacidade 100 usada nos slides de Edmonds-Karp e Dinic.

Vértice	s	a	b	t
Nível BFS	0	1	1	2



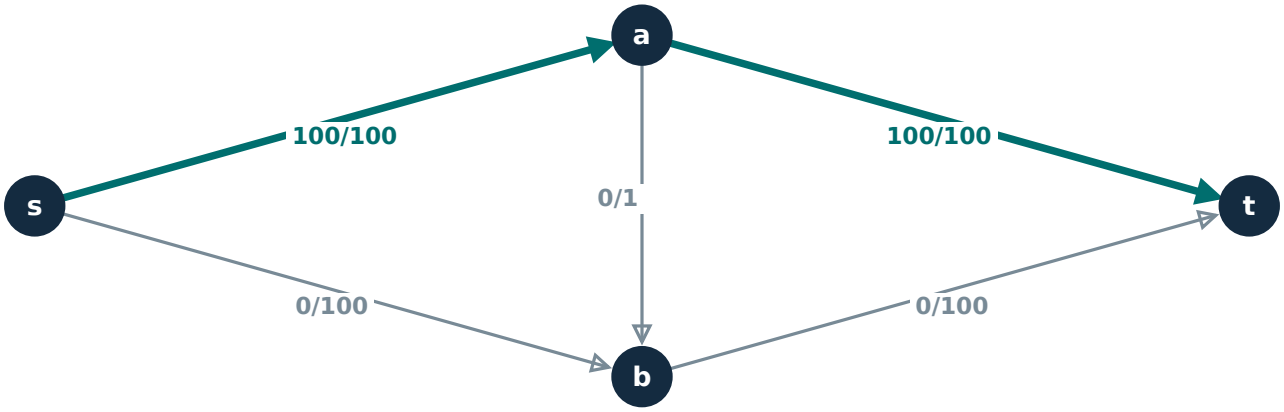
a→b é excluída: vai do nível 1 para o nível 1.

As reversas criadas durante a fase voltam para níveis menores. Permanecem na residual, mas não são elegíveis nessa rede de níveis. Uma fase posterior fará nova BFS.

Zenilton, slides 17, pp. PDF 38–60 e 63.

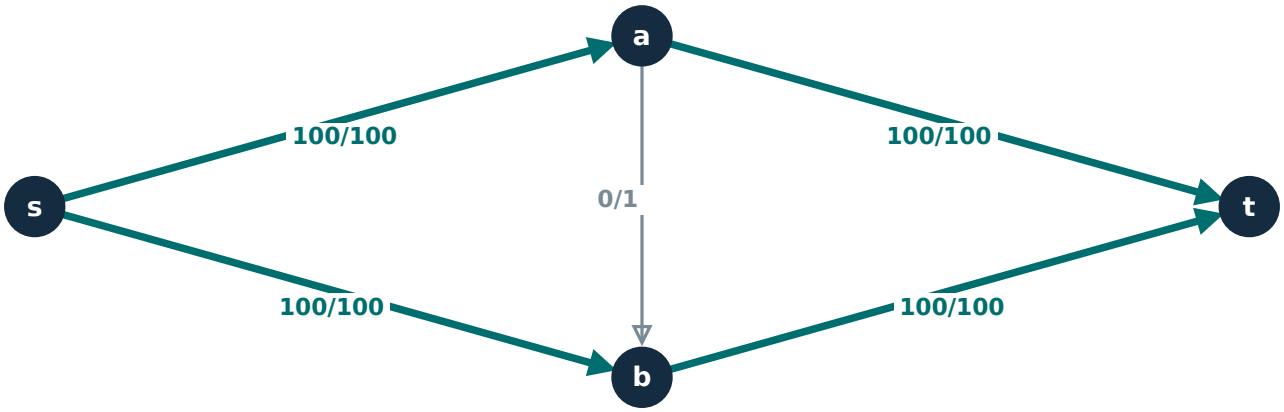
Uma fase, dois envios, um bloqueio

Envio 1 • s → a → t, Δ = 100



Esse caminho saturou, mas s→b→t ainda existe na mesma rede de níveis. Continue a DFS; ainda não é hora de iniciar outra fase.

Envio 2 • s → b → t, Δ = 100



Todo caminho s→t dos níveis tem uma aresta saturada: fluxo adicional de bloqueio=200. A próxima BFS não alcança t. Encerre com fluxo máximo 200.

Zenilton, slides 17, pp. PDF 38–60 e 63.

Três algoritmos de fluxo, lado a lado

Método	Como escolhe / agrupa caminhos	Custo geral
Ford-Fulkerson	Qualquer caminho aumentante.	O(mF), capacidades inteiras.
Edmonds-Karp	BFS: um caminho mínimo em arcos por aumento.	$O(nm^2)$.
Dinic	BFS: níveis; DFS: fluxo de bloqueio por fase.	$O(n^2m)$.

O que permanece igual

Todos mantêm fluxo viável, usam residual e gargalos e permitem cancelar envios pelas reversas. A ausência de caminho aumentante certifica o ótimo.

Neste exemplo, Edmonds-Karp faz duas BFS bem-sucedidas e uma final sem caminho. Dinic encontra os dois caminhos na mesma fase e depois faz uma BFS sem caminho. Em redes maiores, a organização por fases é a diferença central.

Os limites são de pior caso para as implementações usuais com listas de adjacência. Casos especiais podem ter limites melhores. Para acompanhar os slides, use os limites gerais acima.

Zenilton, slides 17, pp. PDF 38–60 e 63.

Fleury: passar sem se prender

Constrói uma trilha que usa cada aresta exatamente uma vez em um grafo não direcionado. Se termina no início, é um circuito euleriano. Vértices podem se repetir; arestas não.

• Verifique conectividade dos vértices de grau não zero. Arestas em componentes separadas impedem um percurso único.

• Todos os graus pares: existe circuito; comece em qualquer vértice de grau positivo.

• Exatamente dois ímpares: existe trilha aberta; comece em um ímpar e termine no outro.

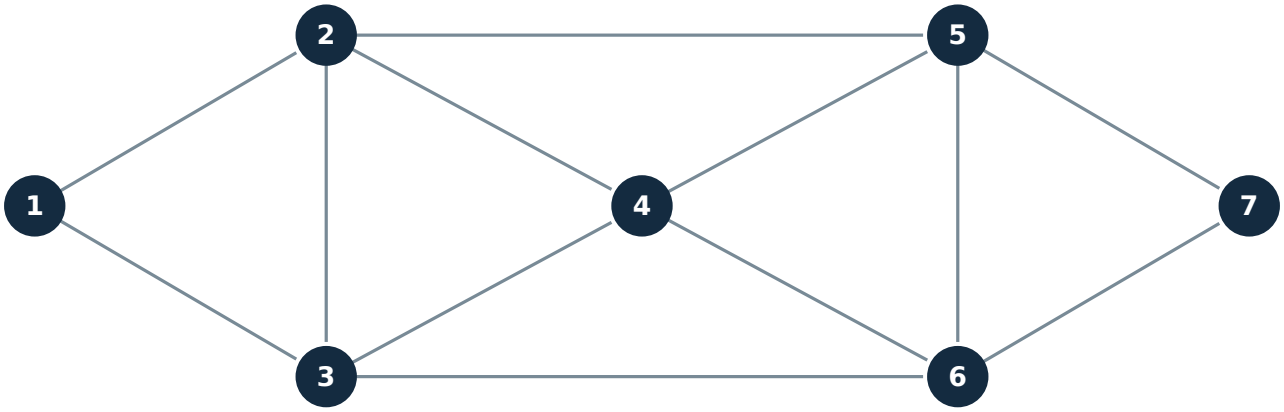
• Mais de dois ímpares: não existe trilha euleriana.

> **A regra dos slides**
>
> No grafo das arestas ainda não usadas, evite uma ponte se houver outra opção. Se houver apenas uma aresta incidente ao atual, use-a. Apague a aresta após percorrê-la.

```
verifique conectividade e graus ímpares
escolha início adequado; copie G para G’
enquanto houver arestas em G’:
    se houver uma única saída do atual: use-a
    senão: escolha uma saída que não seja ponte
    registre o próximo vértice e remova a aresta
```

Zenilton, slides 10, pp. PDF 24–25; West, §1.2, pp. 26–28.

O grafo da aula e o circuito completo



Slide 25. Graus: 1 e 7 têm grau 2; os demais têm grau 4. Todos são pares.

Circuito encontrado

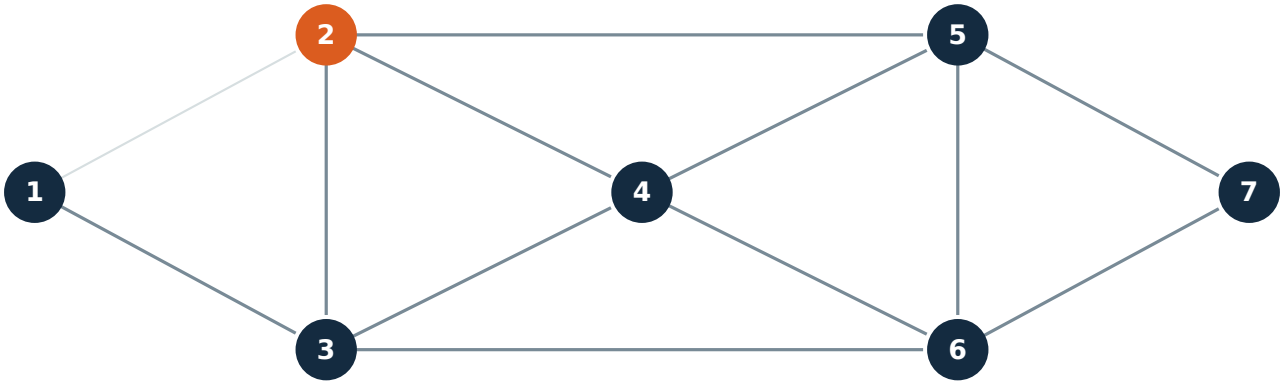
1 → 2 → 3 → 4 → 2 → 5 → 4 → 6 → 5 → 7 → 6 → 3 → 1

Passo	Aresta	Regra no vértice atual
1	1—2	Escolha não ponte
2	2—3	Escolha não ponte
3	3—4	Escolha não ponte
4	4—2	Escolha não ponte
5	2—5	Única opção
6	5—4	Escolha não ponte
7	4—6	Única opção
8	6—5	Escolha não ponte
9	5—7	Única opção
10	7—6	Única opção
11	6—3	Única opção
12	3—1	Única opção

Zenilton, slides 10, pp. PDF 24–25; West, §1.2, pp. 26–28.

Uma ponte aparece: recalcule a decisão

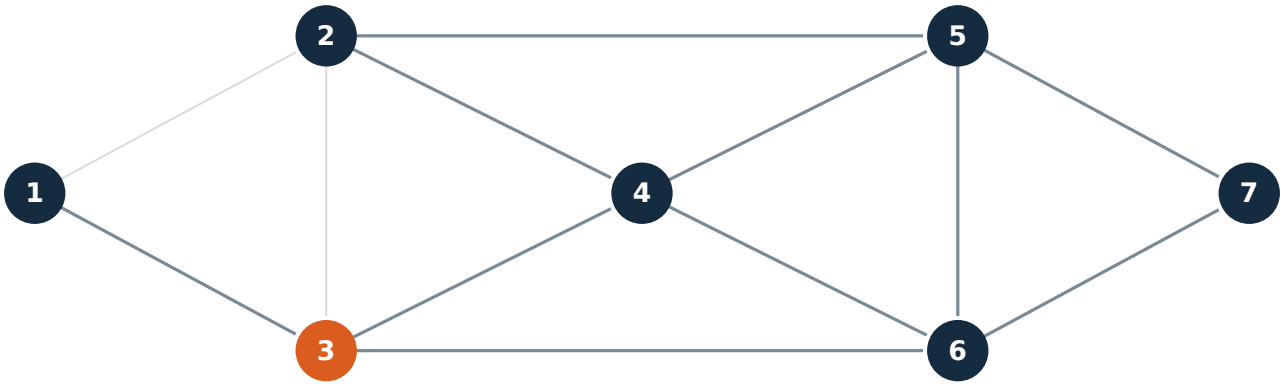
Após o passo 1: atual = 2



Tracejadas claras: já usadas. Escuras: ainda disponíveis.

Depois de 1—2, a aresta 1—3 vira a única ligação de 1 ao resto: agora é ponte. A condição é avaliada no grafo restante.

Após o passo 2: atual = 3



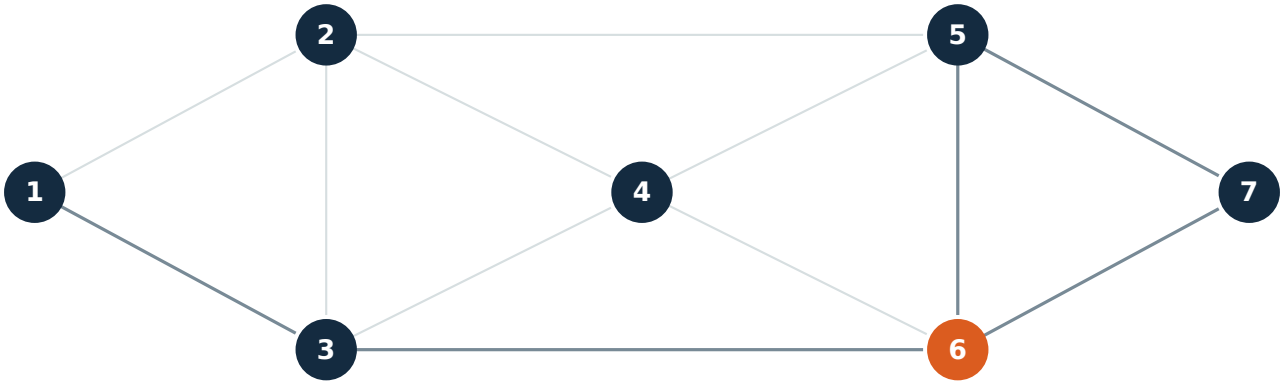
Tracejadas claras: já usadas. Escuras: ainda disponíveis.

Após 1→2→3, escolher 3—1 fecharia o percurso cedo demais. Como há saídas não pontes, escolha 3—4 no próximo passo e deixe 3—1 para o fim.

Zenilton, slides 10, pp. PDF 24–25; West, §1.2, pp. 26–28.

Consuma o restante até fechar o circuito

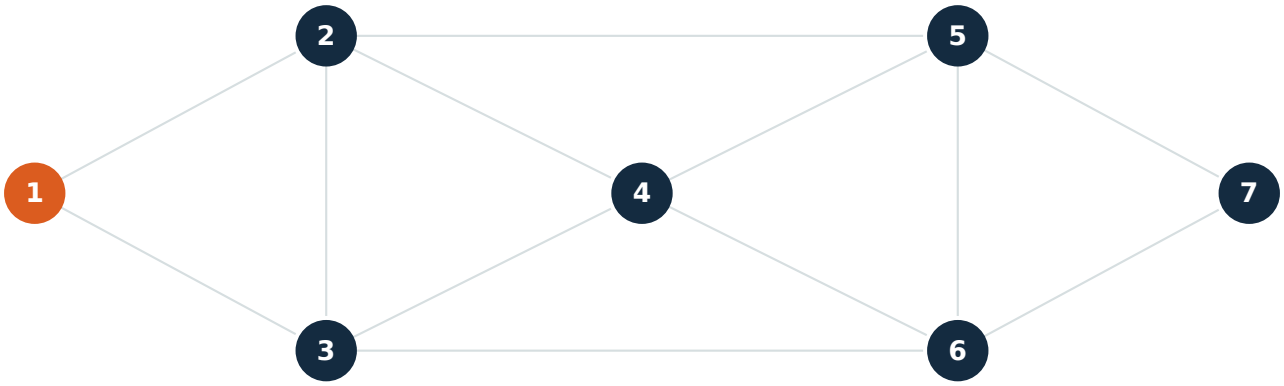
Após o passo 7: atual = 6



Tracejadas claras: já usadas. Escuras: ainda disponíveis.

Usamos 4—6 no passo 7: era a única saída restante de 4. A parte com arestas não usadas continua acessível. O vértice 1 fica para o fim.

Após o passo 12: atual = 1



Tracejadas claras: já usadas. Escuras: ainda disponíveis.

Não restam arestas. Voltamos a 1 depois de usar as 12 arestas exatamente uma vez. O circuito está completo.

Zenilton, slides 10, pp. PDF 24–25; West, §1.2, pp. 26–28.

Como testar ponte e justificar a escolha

Uma aresta é ponte quando sua remoção aumenta o número de componentes. Remova temporariamente a candidata, faça uma busca e teste se as duas pontas ainda se alcançam. Depois recoloque-a, se o teste era só uma consulta.

> **Ponte muda durante a execução**
>
> Uma aresta de um ciclo pode virar ponte quando outras arestas desse ciclo forem usadas. Testar somente as pontes do grafo original não implementa Fleury corretamente.

Evitar a ponte preserva possibilidades de terminar: cruzá-la cedo pode deixar arestas para trás. Se só há uma saída, atravessá-la é necessário e permitido.

Recalculando todas as pontes por DFS a cada etapa: $O(m(n+m))$, frequentemente escrito $O(m^2)$ no suporte conexo. Testar separadamente muitas candidatas com buscas pode ser mais caro. Hierholzer é outro algoritmo e pode ser linear.

Euleriano trata de usar todas as arestas. Hamiltoniano trata de visitar todos os vértices sem repetição no percurso correspondente. Fleury resolve o problema euleriano.

Zenilton, slides 10, pp. PDF 24–25; West, §1.2, pp. 26–28.

Kosaraju: duas DFS separam os grupos

Dois vértices de um digrafo são mutuamente alcançáveis se há caminho de um ao outro e também de volta. Um componente fortemente conexo (CFC) é um conjunto maximal com essa propriedade.

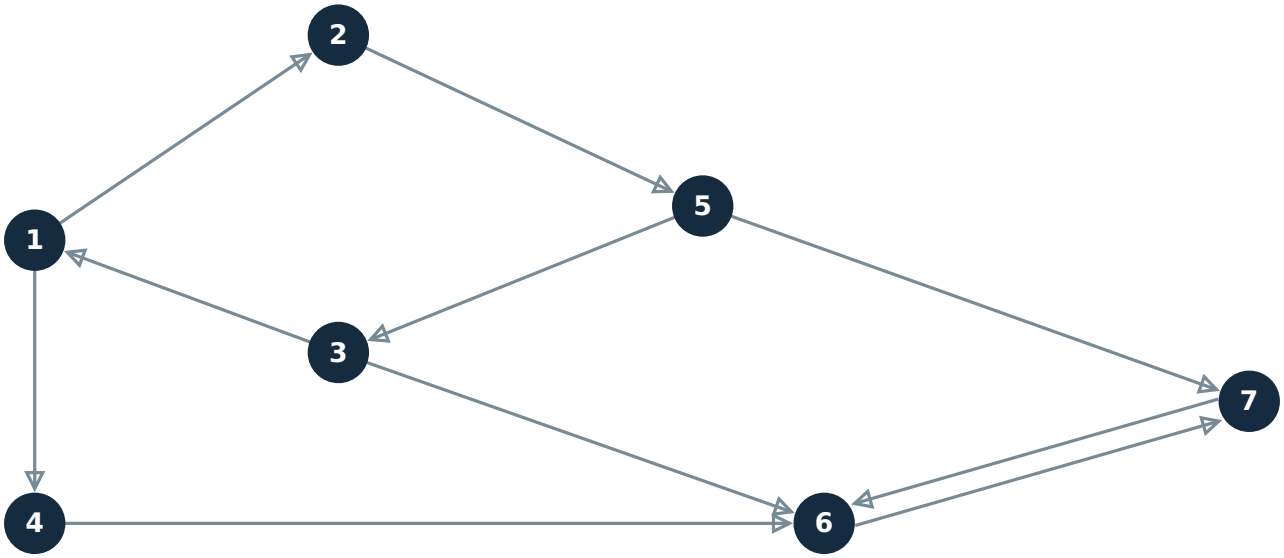
> **A ordem dos seus slides**
>
> DFS em G e tempos de término TT. Inverta as setas para obter G^R. Faça DFS em G^R iniciando raízes por TT decrescente de G. Cada árvore da segunda floresta é um CFC.

```
visitados = vazio; ordem = lista vazia
DFS1(u):
    marque u
    visite os vizinhos não marcados recursivamente
    acrescente u a ordem ao TERMINAR
execute DFS1 em toda raiz não visitada de G
construa G^R; limpe visitados
para u na ordem inversa da lista ordem:
    se u não foi visitado:
        execute DFS em G^R a partir de u
        os novos visitados formam um CFC
```

Tempo O(n+m). Memória O(n+m) incluindo grafo e reverso, mais O(n) de marcas, tempos e pilha. DFS pode precisar de várias raízes em cada passagem.

Zenilton, slides 09, pp. PDF 60–70.

O grafo da aula: ida e volta



O ciclo 1→2→5→3→1 torna {1,2,3,5} mutuamente alcançável. 6 e 7 também se alcançam nos dois sentidos. O vértice 4 alcança 6, mas não recebe retorno de 6.

Componentes esperados

{1,2,3,5}, {4} e {6,7}. Uma seta entre grupos não os funde: é preciso retorno. Um CFC com um único vértice é válido.

Raízes e listas de adjacência serão percorridas em ordem numérica crescente. Isso torna os tempos reproduzíveis. Outra ordem pode gerar TT diferentes e os mesmos componentes.

Zenilton, slides 09, pp. PDF 60–70.

Primeira DFS: descoberta não é término

Vértice	TD (descoberta)	TT (término)
1	1	14
2	2	11
3	4	9
4	12	13
5	3	10
6	5	8
7	6	7

Começamos em 1 e aprofundamos 1→2→5→3→6→7. Ao esgotar as saídas de 7, registramos seu término e retornamos. Depois do ramo de 2, a DFS de 1 visita 4.

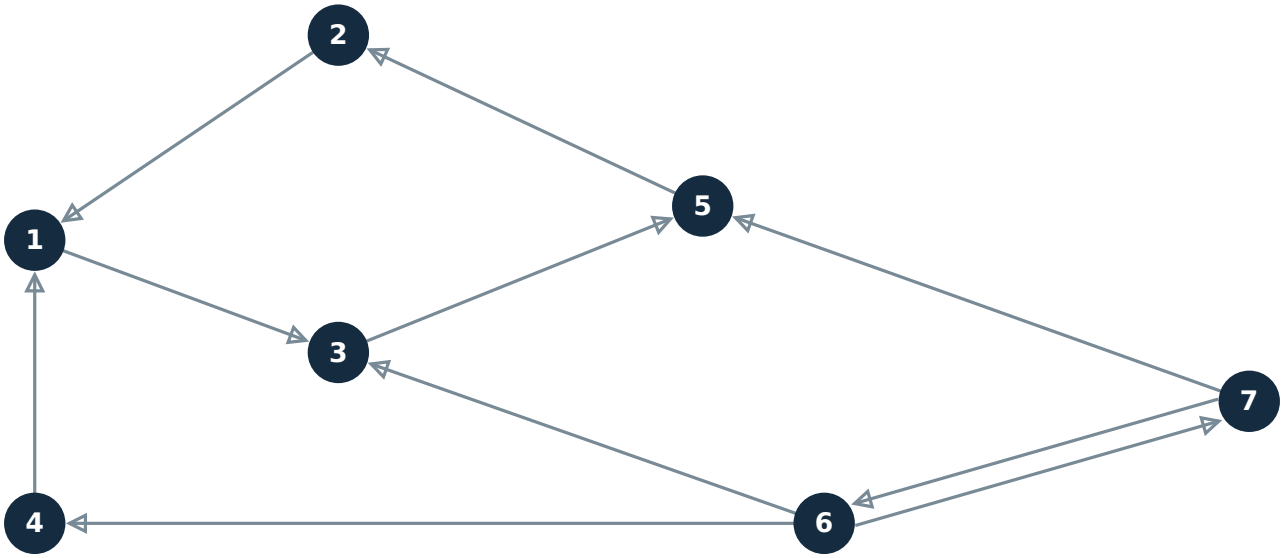
> **Ordem de término**
>
> Ao terminar, empilhamos: 7, 6, 3, 5, 2, 4, 1. Segunda passagem em ordem decrescente de TT: 1, 4, 2, 5, 3, 6, 7.

TD é a entrada no vértice. TT é quando todos os seus vizinhos já foram examinados e voltamos. Usar TD no lugar de TT quebra a garantia.

Alguns vértices da ordem já terão sido visitados pela raiz anterior da segunda passagem. Pule-os: não crie outro componente para eles.

Zenilton, slides 09, pp. PDF 60–70.

Inverta as setas, preserve os vértices



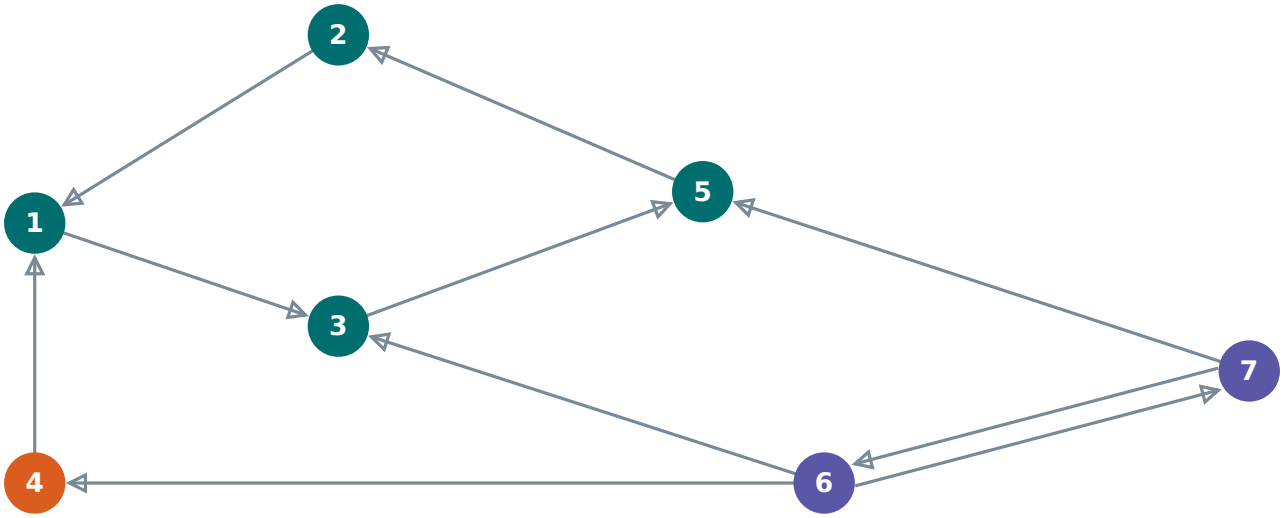
G^R: cada u→v de G virou v→u. Os componentes não dependem de pesos.

Em G	Em G^R
1→2	2→1
1→4	4→1
5→3	3→5
3→6	6→3
6→7 e 7→6	7→6 e 6→7

Limpe as marcas antes da segunda DFS. Use os TT do grafo original, não uma nova ordem arbitrária. Inverta todas as arestas, incluindo as que conectam CFCs.

Zenilton, slides 09, pp. PDF 60–70.

Segunda DFS: uma árvore, um componente



Raiz em G^R	Novos visitados	Conclusão
1	1, 3, 5, 2	Um CFC
4	4	Um CFC
6	6, 7	Um CFC

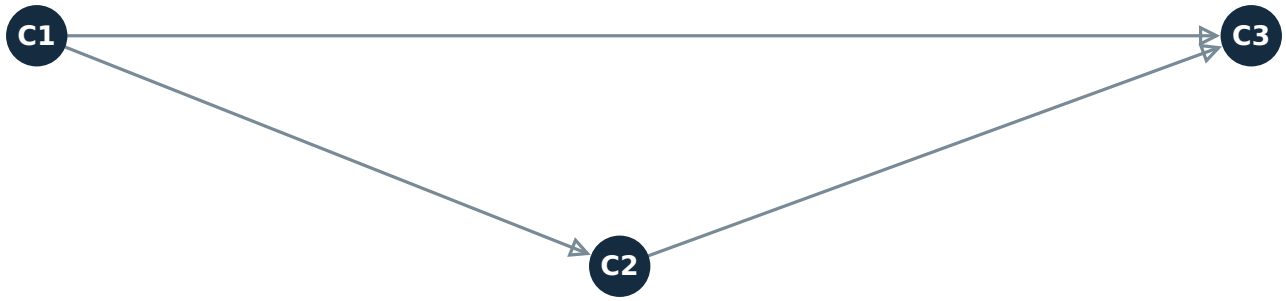
De 1 no reverso, alcançamos 3, 5 e 2. A próxima raiz não marcada é 4; ela não incorpora o grupo anterior, já marcado. Por fim, 6 alcança 7.

Por que não mistura os grupos?

Contraia os CFCs: o grafo resultante é acíclico. A ordem decrescente de término faz a busca no reverso começar por um grupo sem saída para grupos ainda não processados. Cada DFS fica no CFC correto.

Zenilton, slides 09, pp. PDF 60–70.

O resultado e a relação com ciclos



Condensação: C1={1,2,3,5}; C2={4}; C3={6,7}. É um grafo acíclico dirigido (DAG).

Existe ciclo direcionado se algum CFC tiver mais de um vértice ou se um CFC unitário tiver laço. Olhar só o tamanho dos componentes não basta quando laços são permitidos.

> **Checklist**
>
> DFS completa em G; TT ao sair; reversão de todas as setas; marcas limpas; raízes por TT decrescente; uma componente por árvore da segunda DFS.

Ignorar as setas encontra componentes da versão não direcionada, não CFCs. No exemplo, a versão não direcionada é conexa, mas existem três CFCs.

Zenilton, slides 09, pp. PDF 60–70.

Uma frase para lembrar de cada algoritmo

Algoritmo	Decisão para mostrar na prova
Prim	Menor peso cruzando da árvore atual para vértice novo.
Kruskal	Próxima aresta em ordem de peso que não fecha ciclo.
Dijkstra	Finalize o menor d e relaxe suas saídas.
Bellman-Ford	Varra as arestas; repita; teste ciclo negativo em passagem extra.
Floyd-Warshall	Libere intermediário k e atualize todos os pares i,j.
Ford-Fulkerson	Residual, caminho, gargalo, aumento e cancelamento.
Edmonds-Karp	Faça a escolha de Ford-Fulkerson por BFS.
Dinic	BFS de níveis; fluxo de bloqueio; novos níveis.
Fleury	Evite ponte no grafo restante, salvo única saída.
Kosaraju	DFS em G; reverso; DFS por término decrescente.

Como estudar

Refaça os exemplos cobrindo as tabelas. Em cada passo escreva regra, escolha, conta e estado atualizado. Conferir só a resposta final deixa passar erros que custam pontos.

Dez perguntas para testar a compreensão

- 1. Prim: por que b—e=1 não entra no primeiro passo a partir de a?
- 2. Kruskal: que caminho já existente faz e—i=5 fechar ciclo?
- 3. Dijkstra: por que d[e]=2 se a→e tem peso 4?
- 4. Bellman-Ford: melhora na varredura n−1 prova ciclo negativo?
- 5. Floyd-Warshall: que rota justifica D[1,3]=−3?
- 6. Ford-Fulkerson: no segundo aumento, o que significa b→a?
- 7. Edmonds-Karp: BFS minimiza a soma das capacidades?
- 8. Dinic: saturar um caminho já encerra uma fase?
- 9. Fleury: por que não voltar a 1 logo após o passo 2?
- 10. Kosaraju: por que limpar marcas e usar o reverso?

Respostas curtas

1. Ambas as pontas estão fora. 2. e—d—g—i. 3. a→b→e custa 1+1. 4. Não; há verificação adicional após n−1 varreduras. 5. 1→5→4→3. 6. Cancelar 10 de a→b. 7. Não, minimiza número de arcos. 8. Não; bloqueie todos os caminhos dos níveis. 9. Restam arestas e 3—1 é ponte evitável. 10. Marcas limpas permitem a nova exploração; reverso e ordem de término isolam CFCs.

Onde este guia se apoia

Fontes locais consultadas. Páginas PDF contam desde a primeira folha; com animações, diferem do número exibido no canto do slide.

Tema	Arquivo em Grafos/Slides/Zenilton e páginas PDF
Prim / Kruskal	12-AGM-prim-kruskal.pdf: 9–10 e 39–40; grafo 11.
Dijkstra	13-caminho-minimo-dijkstra.pdf: 7–10 e 24–25; grafo 10.
Bellman-Ford	14-caminho-minimo-bellman-ford.pdf: 13–17; grafo 17.
Floyd-Warshall	15-caminho-minimo-floyd-warshall.pdf: 11–22; grafo e D^0 em 22.
Ford-Fulkerson	16-fluxo-maximo-ford-fulkerson.pdf: 8, 14, 29–45 e 57–58.
Edmonds-Karp / Dinic	17-fluxo-maximo-edmonds-karp-dinic.pdf: 20–21, 38–40, 62–63; exemplos 25–36 e 45–60.
Fleury	10-grafos-hamiltonianos-eurelianos-fleury.pdf: regra 24; grafo 25.
Kosaraju	09-conectividade-separabilidade-kosaraju.pdf: 60–70; grafo 62.

A sequência didática preserva os métodos dos slides. Desempates, tempos e ordem de varredura são explicitados aqui; as etapas não pretendem copiar quadro a quadro cada animação.

Livro, apoio complementar e precisão

Douglas B. West • Introduction to Graph Theory, 2ª edição

Grafos/Livro/West_Intro_Graph_Theory_en.pdf é o único livro encontrado na pasta. Edição e seções conferidas no PDF digitalizado.

Seção / páginas impressas	Como complementa o guia
§1.2, 26–28 (PDF 48–50)	Circuitos eulerianos: paridade e conectividade. A sequência de Fleury segue os slides.
§2.3, 95–97 (PDF 117–119)	AGMs: Kruskal, argumento de troca e comentário sobre Prim.
§2.3, 97–99 (PDF 119–121)	Dijkstra, distâncias, predecessores e BFS.
§4.3, 176–180 (PDF 198–202)	Fluxo, caminhos aumentantes, cancelamento, cortes e certificação do ótimo.

Slides complementares de Silvio Jamil F. Guimarães

Grafos/Slides/Silvio: 08-graphs-traversing.pdf, 09-graphs-connectivity.pdf, 10-graphs-shortest-path.pdf, 11-graphs-trees-spanning-trees.pdf e 12-graphs-network-flow.pdf. Fluxo também apresenta caminhos aumentantes, arcos reversos e cortes.

> **Uso das fontes**
>
> Redação própria em português, grafos redesenhados e contas verificadas. Bellman-Ford, Floyd-Warshall, Dinic e a sequência de Kosaraju seguem os slides; não são atribuídos a trechos não conferidos do livro.